

FlatCityBuf: a new cloud-optimised CityJSON format

MSc thesis in Geomatics

FlatCityBuf: a new cloud-optimised CityJSON format

Hidemichi Baba

June 2025

A thesis submitted to the Delft University of Technology in
partial fulfillment of the requirements for the degree of Master of
Science in Geomatics

Hidemichi Baba: *FlatCityBuf: a new cloud-optimised CityJSON format* (2025)

© This work is licensed under a Creative Commons Attribution 4.0 International License.
To view a copy of this license, visit <http://creativecommons.org/licenses/by/4.0/>.

The work in this thesis was carried out in the:



3D geoinformation group
Delft University of Technology

Supervisors: Dr. Hugo Ledoux
Dr. Ravi Peters
Co-reader: Dr. Martijn Meijers

Abstract

Standardising data formats for 3D city models is crucial for semantically storing real-world information as permanent records. CityJSON is a widely adopted OGC standard format for this purpose, and its variant, CityJSON Text Sequences, decomposes large city objects into line-separated objects to enable streaming processing of 3D city model data. However, the shift towards cloud-native environments and the increasing demand for handling massive datasets necessitate more efficient data processing methods across different platforms and on the web. While cloud-optimised data formats such as PMTiles, FlatGeoBuf, Mapbox Vector Tiles have been proposed for vector and raster data, options for 3D city models remain limited. This research aims to explore optimised data formats for CityJSON tailored for cloud-native processing and evaluate their performance and use cases. Specifically, the study implements FlatBuffers for CityJSON, incorporating features like spatial indexing, spatial sorting, indexing with attribute values, and partial fetching via HTTP Range requests. The methodology includes designing a complete binary representation of the CityJSON standard using FlatBuffers, conducting a comprehensive review of existing performance-optimised formats, and benchmarking their performance. Successful implementation of this research will enable end-users to download arbitrary extents of 3D city models efficiently. The research demonstrates that FlatCityBuf achieves superior read performance compared to CityJSONSeq while generally producing smaller file sizes. The approach successfully encoded the entire Netherlands dataset into a single 70GB file containing both spatial and attribute indices, demonstrating scalability for national-scale applications. For developers, the optimised format enables single-file containment of entire areas of interest, simplification of serverless cloud architecture, and accelerated processing by software applications. Ultimately, this work improves the scalability and usability of 3D city models in cloud environments, supporting advanced urban planning and smart city initiatives.

Acknowledgements

This thesis marks the culmination of my journey in the MSc Geomatics program, which would not have been possible without the support and guidance of many individuals and institutions.

First and foremost, I would like to express my sincere gratitude to my primary supervisor, Dr. Hugo Ledoux, for his exceptional guidance, unwavering patience, and continuous encouragement throughout this research. His deep expertise in 3D city models and geospatial data formats has been invaluable, and his critical insights significantly shaped the direction and quality of this work.

I am equally grateful to my secondary supervisor, Dr. Ravi Peters, who made preliminary work on this project and whose pragmatic approach to problem-solving helped me tackle numerous challenges in the development of FlatCityBuf. His constructive feedback consistently pushed me to refine both my ideas and their implementations.

My heartfelt thanks extend to Dr. Martijn Meijers, who served as co-reader for this thesis. His thoughtful comments and perspectives contributed substantially to improving the final manuscript.

I would like to acknowledge Delft University of Technology, particularly the Faculty of Architecture and the Built Environment and the MSc Geomatics program, for providing an exceptional academic environment and resources. The administrative and technical staff deserve special mention for their consistent support throughout my studies.

Beyond academia, I am grateful to my colleagues at Eukarya Inc., who showed remarkable understanding and flexibility as I balanced professional responsibilities with academic pursuits. Their support and accommodation made it possible for me to pursue both paths simultaneously.

My deepest appreciation goes to my family for their unconditional love, encouragement, and belief in me. Their unwavering support has been my anchor throughout this journey, especially during challenging periods.

Finally, a special acknowledgment to Chiharu, my partner, whose patience, understanding, and constant encouragement have been my greatest source of strength. 智春、研究が大変なときも疲れたときもいつも陰ながら応援してくれて本当にありがとう。智春の支えがあって、今日まで頑張ってくることができました。

To everyone who has been part of this journey—thank you.

Hidemichi Baba
Delft, July 4, 2025

...

Contents

List of Figures

List of Tables

Acronyms

1 Introduction

1.1 Problem Statement

Three-dimensional (3D) city models have evolved beyond mere visualisation tools to become fundamental components in diverse application domains. As demonstrated by ?, these models now serve essential functions in urban planning, environmental simulation, emergency response, and numerous other fields, highlighting their critical role in urban environment representation and analysis. The widespread adoption of these models is evidenced by significant national initiatives, such as the Netherlands' comprehensive 3D building database (?), Japan's urban digital twin project (?), the US Open City Model (?), and Switzerland's Swiss-BUILDINGS3D (?). To support this adoption, the CityGML Conceptual Model (?) provides a standardised framework for comprehensive urban environment representation, with implementations including CityGML (?), CityJSON (?), and 3DCityDB (?) achieving substantial adoption in both research and practical applications.

Concurrent with the growth of 3D city models, the geospatial industry has undergone a fundamental shift from desktop-based to cloud-native and web-based **gis!** (**gis!**). Cloud-native **gis!**, as defined by ?, leverages cloud computing infrastructure to deliver geospatial services over networks, enabling ubiquitous access to shared computing resources with minimal management overhead. This paradigm shift offers substantial advantages including global accessibility, multi-user scalability, cross-platform compatibility, and support for diverse applications beyond traditional **gis!** workflows (?). Popular examples such as Google Maps Platform (?) demonstrate how web-based geospatial services have become integral to modern applications, eliminating the need for users to download complete datasets to local machines.

However, this transition introduces specific technical challenges for 3D city model implementations. Cloud-native **gis!** faces inherent constraints including network latency, bandwidth limitations, and the need to serve hundreds or thousands of concurrent users (?). Unlike desktop applications that benefit from high-speed local disk access, web-based systems must efficiently transfer data over networks, necessitating strategies such as data subsetting, streaming, and selective access patterns. Traditional geospatial formats including GeoJSON, Shapefile, and WKT do not inherently support these cloud-optimised access patterns. For example, the **cjseq!** (**cjseq!**) format, which is a variant of CityJSON, is designed to enable streaming processing of 3D city model data, but it still inherits the performance limitations of text-based formats.

The transition towards cloud-native **gis!** introduces specific technical requirements for 3D city model implementations. These requirements encompass scalable processing capabilities, efficient data transfer mechanisms, optimised query performance, and distributed access protocols (?). While CityGML and CityJSON provide comprehensive data models, their text-based implementations often result in slower processing times and increased memory consumption (?). **cjseq!** was developed as a variant of CityJSON to enable streaming processing of 3D city

model data, but it still inherits the performance limitations of text-based formats. Furthermore, although cloud-optimised geospatial formats have emerged to address these challenges, they primarily focus on two-dimensional data, leaving a gap in efficient cloud-native solutions for 3D city models. This gap necessitates the development of specialised data formats that can effectively operate within cloud computing environments while maintaining the semantic richness of 3D city models.

To address these challenges in the cloud-native environment, the geospatial community has developed several optimisation strategies including tiling and partitioning, spatial indexing, data simplification with level-of-detail systems, and binary encoding with compression. These approaches, collectively referred to as *cloud-optimised* geospatial formats (?), enable on-demand access to geospatial data and have proven successful for 2D geospatial applications. However, these optimisation techniques have not been comprehensively applied to 3D city model formats, creating a significant gap in cloud-native solutions for urban digital twin applications.

1.2 Research Objectives

This research investigates the application of efficient data serialisation formats for 3D city models, specifically examining the potential of FlatBuffers (?) as an encoding mechanism for **cjseq!**.

While CityJSONSeq (which will be explained in detail in ??) was designed to enable streaming processing of 3D city models, its text-based format results in suboptimal read performance and lacks efficient indexing strategies for feature querying.

The main research question is: “How can the **cjseq!** encoding be optimised for faster access to features, lower memory consumption, and flexible feature querying in web environments?”

To answer this question, the following sub-questions are addressed:

- What data schema of FlatBuffers is most suitable for encoding all components of **cjseq!**, including geometry templates, materials, extensions, attributes, and CityJSONFeature objects?
- How can feature querying with both spatial and attribute-based operations be achieved within logarithmic time complexity?
- How can subsets of data be retrieved efficiently over the web while maintaining simplicity of server architecture and handling high concurrent request loads that typically challenge traditional database-backed servers?

The following aspects, while relevant to the overall system performance, are not primary focus areas:

- File size reduction, provided the client can efficiently fetch or partially retrieve required data
- Data update and deletion speed, as retrieval speed takes precedence

The resulting proposed data format, called FlatCityBuf, leverages FlatBuffers’ exceptional read performance efficiency and random access capabilities, which are essential for fetching subsets of data without processing the entire file—these advantages will be explained in detail in ??. The proposed methodology combines FlatBuffers’ efficient binary serialisation with **http!**

(**http!**) Range Requests, enabling partial data retrieval over the web while facilitating serverless architectures for enhanced scalability. The investigation aims to address the aforementioned cloud-native requirements while maintaining the semantic richness of CityJSON’s data model. Notably, the research prioritises read performance over update capabilities, as read operations predominate in typical use cases. Furthermore, while file size optimisation remains relevant, it is considered secondary to query efficiency and partial data accessibility. The technical implementation strategy and preliminary findings are detailed in ??, while the evaluation of cloud-optimised formats is presented in ??.

1.3 Scope of the Research

Since the primary focus of the research is to explore the potential of FlatBuffers as an encoding mechanism for CityJSONSeq and achieve faster and flexible feature querying, the scope of the research is limited to the following:

- Define the data specification of the proposed new encoding format, FlatCityBuf.
- Implement a Rust library for encoding and decoding operations for FlatCityBuf, with **wasm!** (**wasm!**) bindings for web-based decoding.
- Support spatial querying and attribute-based querying to achieve log-time complexity for feature retrieval.
- Demonstrate how the proposed encoding format can be used over the web with **http!** Range Requests.
- Evaluate the performance of the proposed encoding format compared to the other encoding formats.

On the other hand, the following aspects are considered secondary and are not within the scope of the research:

- Implementing the library with other programming languages than Rust such as Python or JavaScript.
- Implementing the library to encode with other serialisation frameworks such as Parquet (?) or Protocol Buffers (?).
- Optimising the encoding format for write operations.

1.4 Structure of the Thesis

This thesis is organised into the following chapters:

?? establishes the fundamental knowledge of serialisation formats, algorithms, and indexing strategies necessary for understanding the proposed solution.

?? provides a review of the relevant literature, focusing on cloud-optimised geospatial formats and serialisation frameworks. It presents a comprehensive analysis of existing cloud-optimised geospatial formats and their characteristics.

?? details the methodology used to achieve the research objectives.

1 Introduction

It explains the data specification of FlatCityBuf and other technical components designed to address the research questions.

?? presents the research findings, including file size comparisons, local benchmark results, and web-based performance evaluations.

?? elaborates on the results, discusses the implications of the research, and identifies potential applications and limitations.

?? concludes the thesis by answering the research questions and summarising the contributions of the research.

2 Theoretical background

2.1 Strategies for Cloud-native GIS

The benefits and challenges of cloud-native GIS are explained in ???. This section explains the general strategies employed for cloud-native optimisation.

To address cloud-native GIS challenges while leveraging its advantages, several strategies are commonly employed:

- **Tiling and partitioning:** Large datasets are divided into small, manageable tiles or chunks. Tile Map Service (TMS) exemplifies this approach by dividing maps into square tiles for each zoom level. As users zoom in or out, new tiles are loaded and displayed, with clients loading only necessary tiles. Figure ?? shows an example of TMS, a standard for tiling geospatial data (?). This technique requires either pre-generated tiled data or dynamic tile generation by servers like GeoServer (?). Both raster and vector data are commonly tiled. This approach is not limited to 2D geospatial data; it is also used for 3D city model data. For instance, 3DBAG divides the entire Netherlands into tiles for selective download (?) and PLATEAU publishes tiled datasets as open data (?).
- **Spatial indexing:** Spatial indexing techniques index geospatial data by spatial properties. R-trees (?), quadtrees (?), and KD-trees (?) are commonly used indexing structures. These indices enable clients to quickly locate data within specified extents, find nearest neighbours, or perform other spatial queries. This technique requires pre-built spatial indices that must be updated when data changes.
- **Data simplification and Level of Detail:** Also known as generalisation, this reduces geospatial data complexity for visualisation, especially at smaller scales. This technique is often combined with tiling and partitioning. For example, Cesium 3D Tiles (?) implements level-of-detail systems where higher zoom levels load more detailed data. GDAL's Mapbox Vector Tile driver also supports geometry simplification for smaller scales (?).
- **Binary encoding and compression:** This technique reduces geospatial data size, resulting in faster download and display times. While compression reduces file size, it requires client-side decompression, creating a trade-off between file size and decompression time. For example, glTF 2.0 (?), used internally by Cesium 3D Tiles, supports Draco compression (?). GeoParquet (?) also supports various compression algorithms.

When incorporating these strategies, the optimisation approaches that enable on-demand access to geospatial data are collectively referred to as *cloud-optimised* formats (?). The properties and related work of these cloud-optimised geospatial formats are discussed in detail in ???.

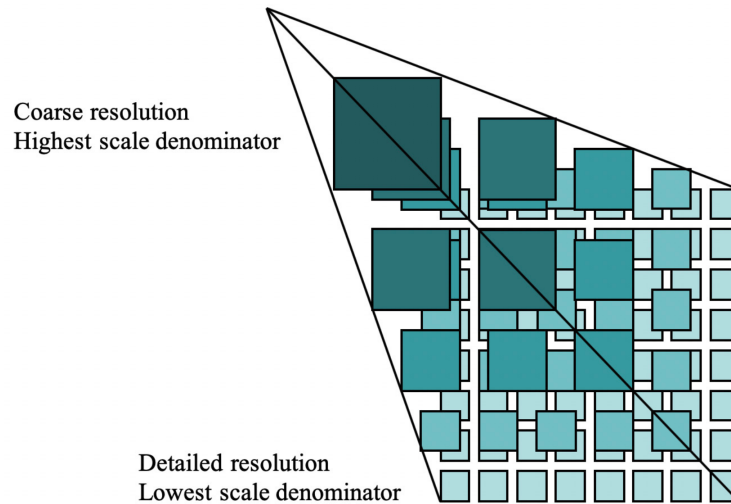


Figure 2.1: Tile Map Service (TMS) standard for tiling geospatial data, derived from ?

2.2 Binary files

A binary file represents a fundamental approach to data storage that uses sequences of bits rather than human-readable text. Unlike plain text files that consist of characters interpreted through common character sets like ASCII, binary files store data in a format optimised for machine processing (?).

Binary encoding offers several key advantages: superior storage efficiency through data compression and faster program execution. Binary files are commonly used for images, audio, executable programs, and compressed data. However, binary encoding presents notable challenges. Binary files are not human-readable, making debugging, correction, or modification complex without specialised tools. Many binary systems are proprietary and platform-specific, creating portability issues and potential long-term accessibility problems. The Unix philosophy advocates for plain text storage when practical, emphasising that text serves as a universal interface enabling efficient program interoperability (?).

In geospatial domains, prominent examples of text-based formats include GeoJSON (?) and Geography Markup Language (GML) (?) as text representations for general geospatial data. For 3D city model data specifically, CityGML (?) and CityJSON (?) (including its streaming variant CityJSONSeq (?)) represent the primary text-based standards. Conversely, binary geospatial formats include GeoPackage (?) and GeoTIFF (?). Notably, no widely adopted standard binary formats currently exist for 3D city model data, representing a gap that this research aims to address.

2.3 WebAssembly

wasm! is a low-level assembly-like language with a compact binary format that enables near-native performance execution in modern web browsers (?). Standardised by the ??, **wasm!**

provides a compilation target for languages such as C/C++, C#, and Rust, allowing code written in these languages to run efficiently on the web platform.

wasm! is designed to complement and run alongside JavaScript rather than replace it. Through the WebAssembly JavaScript APIs, developers can load **wasm!** modules into JavaScript applications and share functionality between the two environments (?). This interoperability enables developers to leverage **wasm!**'s performance characteristics for computationally intensive tasks while maintaining JavaScript's expressiveness and flexibility for application logic and user interface development.

A key advantage of **wasm!** is its ability to enable code reuse across platforms. Developers can compile existing codebases written in languages like C/C++, Rust, and C# for browser deployment, eliminating the need to rewrite performance-critical components in JavaScript.

In this research, **wasm!** enables the compilation of Rust libraries for 3D city model processing to run with near-native performance in web browsers. This approach bridges the performance gap between desktop and web-based geospatial applications while maintaining the accessibility and cross-platform benefits of web deployment.

2.4 Row-based and column-based data storage

Data storage systems can be broadly categorised into two fundamental approaches: row-based and column-based storage. Row-based storage stores consecutive rows of a table sequentially, while column-based storage stores consecutive columns of a table sequentially (?).

To illustrate this distinction, consider the following example table:

```

1 id , city , country
2 1 , Tokyo , Japan
3 2 , London , UK
4 3 , Amsterdam , Netherlands

```

Listing 2.1: Example table

Row-based storage organises the data sequentially by rows:

```

1 1 , Tokyo , Japan , 2 , London , UK , 3 , Amsterdam , Netherlands

```

Listing 2.2: Row-based storage

Column-based storage organises the data sequentially by columns:

```

1 1 , 2 , 3 , Tokyo , London , Amsterdam , Japan , UK , Netherlands

```

Listing 2.3: Column-based storage

These different storage approaches exhibit distinct performance characteristics depending on the query patterns. Row-based approaches perform well for single-row searches, such as querying the record for Amsterdam. Conversely, column-based approaches excel at analytical queries that involve aggregating or filtering columns (?).

? provides a comprehensive comparison of column-stores versus row-stores, highlighting that column-based storage offers advantages in terms of storage efficiency and query performance

for analytical workloads, while row-based storage is more suitable for Online Transaction Processing (OLTP) systems that require frequent lookups and updates.

In the context of geospatial data, column-based storage is particularly well-suited for analytical queries such as calculating the average height of buildings in a city or filtering buildings by construction year. However, row-based storage remains preferable for operations that require accessing entire records, such as retrieving all attributes of a specific building, as well as for frequently updated datasets or transactional systems where individual records are modified regularly.

2.5 CPU Caches

Modern computing systems face a fundamental challenge to software performance due to the disparity between the speed of **cpu!** (**cpu!**) and the latency of main memory. **cpu!** operates at clock speeds that far exceed the access speeds of dynamic random-access memory (DRAM), which typically serves as main memory. Consequently, the **cpu!** frequently idles while waiting for data from the slower memory subsystem. To mitigate this performance bottleneck, **cpu!** caches were developed. These caches utilise a small amount of very fast Static RAM (SRAM) to store temporary copies of data and instructions that are likely to be used again soon (?).

The effectiveness of **cpu!** caches hinges on two fundamental principles of program behaviour: temporal locality and spatial locality. Temporal locality refers to the tendency of a program to access data that has been recently accessed again in the near future. Spatial locality refers to the tendency of a program to access data that is located nearby in memory to previously accessed data. In other words, small chunks of data that have been accessed recently or are located nearby in memory are likely to be accessed again soon. Modern processors employ multiple levels of caches to maximise the effectiveness of these principles. This hierarchy typically consists of L1, L2, L3 caches, and main memory. The lower levels of the cache hierarchy are faster and smaller, while the higher levels are slower and larger (?).

Data is not loaded into caches byte by byte. Instead, it is loaded in blocks called cache lines, typically 64 bytes in size. This design reduces the number of separate memory transactions and effectively amortises the substantial latency involved in accessing main memory (?).

The performance of a cache is fundamentally determined by its hit rate. A cache hit occurs when the requested data is found within the cache, resulting in very fast access. In contrast, a cache miss occurs when the requested data is not found within the cache, necessitating a much slower retrieval from a higher-level cache or, in the worst case, main memory (?).

Understanding these aspects of cache behaviour is crucial when designing binary data formats, as cache-friendly data layouts can significantly improve application performance.

2.6 Serialisation and Deserialisation

Before discussing the specific techniques used in the FlatCityBuf format, it is important to understand the general principles of serialisation and deserialisation.

The terminology for data conversion processes varies across different programming ecosystems. Terms such as serialisation, pickling, marshallng, and flattening are often used interchangeably,

though with subtle differences depending on the context. [1] describes it from an object-oriented perspective as converting objects in memory (in a data structure) to a storable or transmittable format on disk. [1] refers to this process as "pickling" in the Python ecosystem. For clarity in this thesis, we adopt the definition provided by [2]:

"Serialisation is the process of translating a data structure into a bit-string (a sequence of bits) for storage or transmission purposes."

Deserialisation is the reverse process of serialisation, where the bit-string is converted back into the original data structure (in memory).

2.7 Zero-copy

Zero-copy is a technique used to avoid copying data from one memory location to another. The term "Zero-copy" is used in many contexts of computer science, [3] and [4] provide a detailed explanation of the concept.

In conventional I/O operations, data typically traverses multiple memory regions, each requiring a separate copy operation:

- Data is copied from storage devices into kernel buffer cache
- From kernel buffer, data is copied to user-space application buffers
- For network transmission, data may be copied again to network buffers

This multi-stage copying introduces significant overhead, particularly for large datasets or high-throughput applications. Each copy operation consumes **cpu** cycles, memory bandwidth, and increases latency ([5]). For applications working with large 3D city models, this overhead can substantially degrade performance.

Zero-copy approaches optimise this data path by eliminating unnecessary copy operations. While "zero-copy" as a term suggests complete elimination of copying, in practice, different techniques achieve varying degrees of copy reduction:

- **Memory-mapped I/O:** Maps files directly into process address space using `mmap()`, allowing direct access without explicit `read()` operations and avoiding data copying between kernel and user space.
- **In-place parsing:** Processes data structures without creating intermediate copies, enabling direct access to serialised data in its original memory location.
- **Zero-copy system calls:** `sendfile()` and `splice()` enable direct kernel-level data transfer between file descriptors and sockets without user-space copying.
- **Shared memory:** Provides common address space for inter-process communication, eliminating the need for data copying between processes.

Modern serialisation formats like FlatBuffers implement zero-copy through carefully designed memory layouts that allow direct access to serialised data without requiring a separate deserialisation step. This approach is particularly valuable for geospatial applications that routinely handle large datasets and benefit from avoiding the memory overhead of traditional parse-then-access patterns.

2.8 Endianness

Endianness (or "byte-order") refers to the order in which bytes are stored in memory when representing multi-byte values. The terminology was introduced by ?.

In computing, endianness becomes significant when multi-byte data types (such as 16-bit integers or 32-bit floats) must be stored in memory or transmitted across networks. There are two primary byte ordering systems:

- **Little-endian:** Stores the least significant byte at the lowest memory address, followed by increasingly significant bytes. This is the ordering used by Intel processors that dominate desktop and server computing. For example, the 32-bit integer `0x12345678` would be stored in memory as 4 bytes: `0x78`, `0x56`, `0x34`, `0x12`.
- **Big-endian:** Stores the most significant byte at the lowest memory address. This approach is often called "network byte order" because Internet protocols typically require data to be transmitted in big-endian format. For example, the same 32-bit integer `0x12345678` would be stored as `0x12`, `0x34`, `0x56`, `0x78`.

A useful analogy is date notation: little-endian resembles the European date format (31 December 2050), while big-endian resembles the ISO format (2050-12-31), with the most significant part (year) first (?).

2.9 Binary Search

Binary search is a fundamental algorithm for finding elements in a sorted array. The classic implementation follows a simple approach: compare the search key with the middle element of the array, then recursively search the left or right half depending on the comparison result (?).

The time complexity of binary search is logarithmic—the height of the implicit binary search tree is $\log_2(n)$ for an array of size n . While this is theoretically efficient, the actual performance suffers when implemented on modern hardware due to memory access patterns. Each comparison requires the processor to fetch a new element, potentially causing a cache miss (cache miss is explained in ??). In the worst case, the number of memory read operations will be proportional to the height of the tree, with each read potentially requiring access to a different cache line or disk block (?).

This inefficiency is particularly problematic when binary search is implemented on external memory or over **http!**, where each access incurs significant latency. The sorted array representation with binary search does not take advantage of **cpu!** cache locality, as consecutive comparisons frequently access distant memory locations.

2.9.1 Eytzinger Layout

While preserving the same algorithmic idea as binary search, the Eytzinger layout (also known as a complete binary tree layout or level-order layout) rearranges the array elements to match the access pattern of a binary search (?). Instead of storing elements in sorted order, it places

them in the order they would be visited during a level-order traversal of a complete binary tree.

This layout significantly improves memory access patterns. When the array is accessed in the sequence of a binary search operation, adjacent accesses often refer to elements that are in the same or adjacent cache lines. This layout also proves beneficial for managing data fetched over networks, as consecutive elements accessed during binary search are more likely to be retrieved in the same **http!** range request, thereby reducing the number of network roundtrips and overall latency. This spatial locality enables effective hardware prefetching, allowing the **cpu!** to anticipate and load required data before it is explicitly accessed, thus reducing latency (?).

?? shows how the layout appears when applied to binary search. ?? shows that the algorithm starts from the first element and then jumps to either $2k$ or $2k + 1$ depending on the comparison result. The heatmap represents the expected frequency of comparisons for search (the closer to the top, the more frequent the comparison).

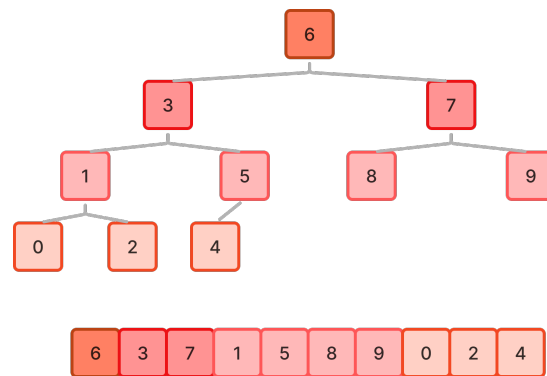


Figure 2.2: Eytzinger layout as conceptual representation as tree and actual data layout (modified from ?)

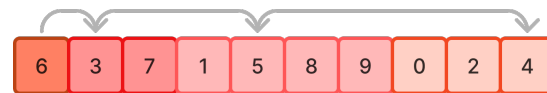


Figure 2.3: Binary search traversal pattern in Eytzinger layout (modified from ?)

2.10 s+tree! (s+tree!)

2.10.1 B-Tree/B+Tree Layout

While the Eytzinger layout improves cache utilisation for binary search, the number of memory read operations remains proportional to the height of the tree— $\log_2(n)$ for n elements. This is still suboptimal for large datasets, especially when the access pattern involves disk **io!** (**io!**) or remote data access (?).

B-Trees and their variants address this limitation by storing multiple keys in each node, effectively reducing the height of the tree. In a B-Tree of order k (where each node can contain

2 Theoretical background

up to $k - 1$ keys), the height of the tree is reduced from $\log_2(n)$ to $\log_k(n)$. This represents a reduction factor of $\log_k / \log_2 = \log_2(k)$ times compared to a binary search tree.

The key insight is that fetching a single node still takes roughly the same time regardless of whether it contains one key or multiple keys, as long as the entire node fits into a single memory block or disk page. By packing multiple keys into each node, B-Trees significantly reduce the number of disk or memory accesses required to locate an element.

B+Trees are a variant of B-Trees specifically optimised for range queries and sequential access patterns. In a B+Tree:

- Internal nodes contain up to B keys that serve as routing information, with each key associated with one of the $(B + 1)$ pointers to child nodes. Each key at position i represents the smallest key in the subtree pointed to by the $(i + 1)$ -th child pointer.
- Leaf nodes store the actual data with up to B key-value pairs and include a pointer to the next leaf node, enabling efficient sequential traversal for range queries.

This linked structure of leaf nodes enables B+Trees to efficiently support range queries by traversing from one leaf to the next without needing to return to higher levels of the tree.

As the figure ?? shows, the B+Tree has pointers to the next leaf node, which enables efficient sequential traversal for range queries. On the other hand, the B+Tree has duplicate keys in the internal nodes, which is not the case for the B-Tree.

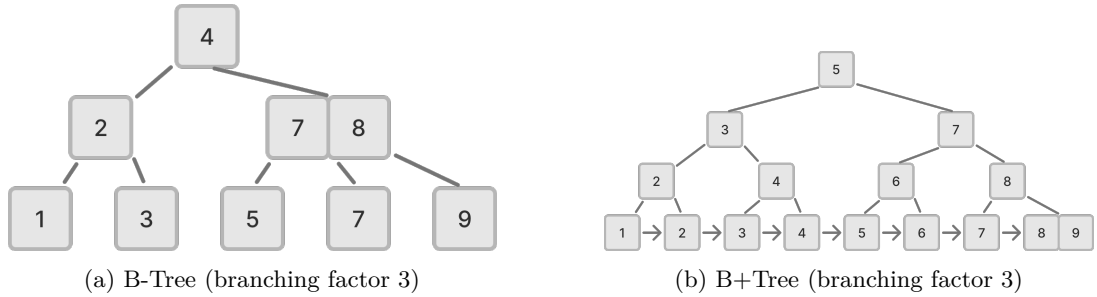


Figure 2.4: B-Tree and B+Tree

2.10.2 s+tree!

The S+Tree (Static B+Tree), introduced by ?, builds upon the B+Tree concept but is specifically designed for static datasets where the tree structure never changes after construction. Unlike traditional B+Trees that use explicit pointers between nodes, the S+Tree uses an implicit structure where child positions are calculated mathematically. This is possible because:

- The tree is constructed once and never modified (static)
- The number of elements is known in advance
- The tree can be maximally filled with no empty slots
- Child positions follow a predictable pattern based on the block size

For a **s+tree!** with block size B , a node with index k has its children at indices calculated by a simple formula: $\text{child}_i(k) = k \cdot (B + 1) + i + 1$ for $i \in [0, B]$ (?). This eliminates the need to store and fetch explicit pointer values, further reducing memory usage and improving cache efficiency.

The S+Tree layout aligns with modern hardware characteristics where:

- The latency of fetching a single byte is comparable to fetching an entire cache line (64 bytes)
- Disk and network **io!** operations have high initial latency but relatively low marginal cost for additional bytes
- **cpu!** cache lines typically hold multiple array elements (e.g., 16 integers in a 64-byte cache line)

By loading a block of B elements at once and performing a local search within that block, S+Trees reduce the total number of cache misses or disk accesses to $\log_B(n)$ instead of $\log_2(n)$ —a significant reduction for large datasets.

The S+Tree layout achieves up to $15\times$ performance improvement over standard binary search implementations while requiring only 6-7% additional memory (?). This makes it particularly valuable for applications that perform frequent searches on large, relatively static datasets, especially when accessed over high-latency connections. For more detailed implementation strategies of S+Tree, ? provides comprehensive explanations and practical considerations.

2.11 FlatBuffers Framework

FlatBuffers, developed by [Google](#), is a cross-platform serialisation framework designed specifically for performance-critical applications with a focus on memory efficiency and processing speed. Unlike traditional serialisation approaches, FlatBuffers implements a zero-copy deserialisation mechanism that enables direct access to serialised data without an intermediate parsing step ([?](#)), as discussed in [??](#). This characteristic is particularly advantageous for large geospatial datasets, where parsing overhead can significantly impact performance (e.g., JSON parsing).

2.11.1 Schema-Based Serialisation

FlatBuffers employs a strongly typed, schema-based approach to data serialisation. The workflow involves:

1. Definition of data structures in schema files with the `.fbs` extension
2. Compilation of schema files using the FlatBuffers compiler (`flatc`) to generate language-specific code for data access
3. Implementation of application logic using the generated code

This schema-first approach enforces data consistency and type safety, which is essential to be processed in various programming languages and environments. The generated code provides memory-efficient access patterns to the underlying binary data without requiring full deserialisation. FlatCityBuf utilises this capability to achieve a balance between parsing speed and storage efficiency.

The FlatBuffers compiler supports code generation for multiple programming languages, including C++, Java, C#, Go, Python, JavaScript, TypeScript, Rust, and others, facilitating cross-platform interoperability ([?](#)). This extensive language support enables developers to work with FlatBuffers data in their preferred environment. For FlatCityBuf, Rust was selected as the primary implementation language due to its performance characteristics, memory safety guarantees, and excellent support for `wasm!` compilation, which is beneficial for web-based deployment scenarios.

2.11.2 Data Type System

FlatBuffers provides a comprehensive type system that balances efficiency and expressiveness ([?](#)):

- **Tables:** Variable-sized object containers that support:
 - Named fields with type annotations
 - Optional fields with default values
 - Schema evolution through backward compatibility
 - Non-sequential field storage for memory optimisation
- **Structs:** Fixed-size, inline aggregates that:
 - Require all fields to be present (no optionality)

- Are stored directly within their containing object
- Are less flexible than tables but provide faster access
- Optimise memory layout for primitive types
- **Scalar Types:** FlatBuffers supports a comprehensive range of primitive data types as shown in ??.

Table 2.1: FlatBuffers Scalar Data Types

Category	Type	Description
8-bit integers	<code>byte</code>	Signed 8-bit integer (int8)
	<code>ubyte</code>	Unsigned 8-bit integer (uint8)
	<code>bool</code>	Boolean value
16-bit integers	<code>short</code>	Signed 16-bit integer (int16)
	<code>ushort</code>	Unsigned 16-bit integer (uint16)
32-bit values	<code>int</code>	Signed 32-bit integer (int32)
	<code>uint</code>	Unsigned 32-bit integer (uint32)
	<code>float</code>	32-bit floating-point number
64-bit values	<code>long</code>	Signed 64-bit integer (int64)
	<code>ulong</code>	Unsigned 64-bit integer (uint64)
	<code>double</code>	64-bit floating-point number

- **Complex Types:** FlatBuffers provides advanced data structures as outlined in ??.

Table 2.2: FlatBuffers Complex Data Types

Type	Description
<code>[T]</code>	Vectors (single-dimension arrays) of any supported type
<code>string</code>	UTF-8 or 7-bit ASCII encoded text with length prefix
References	References to other tables, structs, or unions

- **Enums:** Type-safe constants mapped to underlying integer types
- **Unions:** Tagged unions supporting variant types

2.11.3 Schema Organisation Features

In addition to the data type system, FlatBuffers provides several key features for organising complex schemas:

- **Namespaces** (`namespace FlatCityBuf;`) create logical boundaries and prevent naming collisions
- **Include Mechanism** (`include "header.fbs";`) enables modular schema design across multiple files
- **Root Type** (`root_type Header;`) identifies the primary table that serves as the entry point for buffer access

These features were essential for FlatCityBuf’s implementation, enabling modular schema development with separate root types for header and feature components while maintaining consistent type definitions across files.

2.11.4 Binary Structure and Memory Layout

FlatBuffers organises serialised data in a flat binary buffer with the following characteristics:

- **Zero-copy access** through a carefully designed memory layout that allows direct access to serialised data without intermediate parsing
- **Vtable-based field access** where each table starts with an offset to its vtable, enabling efficient field lookup and schema evolution. ?? shows an example of a vtable.
- **Little-endian encoding** for all scalar values, with automatic conversion on big-endian platforms
- **Offset-based references** for all non-inline data (tables, strings, vectors), allowing efficient navigation within the buffer

```

1  vtable (AnnotatedBinary.Bar):
2  +0x00A0 | 08 00      | uint16_t   | 0x0008 (8)   | size of this vtable
3  +0x00A2 | 13 00      | uint16_t   | 0x0013 (19)  | size of referring table
4  +0x00A4 | 08 00      | VOffset16  | 0x0008 (8)   | offset to field 'a' (id: 0)
5  +0x00A6 | 04 00      | VOffset16  | 0x0004 (4)   | offset to field 'b' (id: 1)

```

Listing 2.4: Vtable of AnnotatedBinary.Bar sourced from ?

For complex data structures like 3D city models, FlatBuffers allows for modular schema composition through file inclusion. This capability enabled the separation of FlatCityBuf’s schema into logical components (`header.fbs`, `feature.fbs`, `geometry.fbs`, etc.) while maintaining efficient serialisation. In our implementation, the `Header` and `CityFeature` tables serve as root types that anchor the overall data structure.

3 Related Work

This chapter reviews the pertinent literature relevant to the optimisation of CityJSON for cloud-native environments. It highlights advancements and identifies existing gaps that this research aims to address.

3.1 Cloud-Optimised Geospatial Formats

Cloud-optimised geospatial formats are data formats that are optimised for cloud environments by enabling efficient on-demand access to geospatial data in contrast to traditional GIS formats (?). ? defines four advantages of cloud-optimised geospatial formats:

- **Reduced Latency:** Facilitates partial data retrieval and processing without necessitating complete file downloads.
- **Scalability:** Supports parallel operations through metadata-driven access mechanisms within cloud storage systems.
- **Flexibility:** Offers advanced query capabilities for selective data access.
- **Cost-Effectiveness:** Optimises storage and transfer expenditures through efficient access patterns.

These include Cloud Optimised GeoTIFF, Cloud Optimised Point Cloud, GeoParquet, PMTiles, and FlatGeobuf. Although not mentioned by ?, 3D Tiles (?) and Mapbox Vector Tiles (?) can also be considered as cloud-optimised geospatial formats since they were developed by Web GIS companies Cesium and Mapbox respectively.

3.2 CityGML, CityJSON and Its Enhancements

3.2.1 CityGML

CityGML is an **ogc!** (**ogc!**) standard (?) that defines a comprehensive data model for representing 3D city models. The standard encompasses both geometric properties and rich semantic information through a modular structure. From version 3.0.0, CityGML separates its conceptual model from its encoding standard. ?? shows an overview of its modules. The conceptual model defines the semantics and data model through a Core module and eleven thematic extension modules (Building, Bridge, Tunnel, Construction, CityFurniture, CityObjectGroup, LandUse, Relief, Transportation, Vegetation, and WaterBody). Additionally, five extension modules (Appearance, PointCloud, Generics, Versioning, and Dynamizer) provide specialised modelling capabilities applicable across all thematic modules. The encoding standard uses GML application schema for the Geography Markup Language (GML) (?) to encode

the data. This modular design allows implementations to support specific subsets of modules based on their application requirements, ensuring flexibility while maintaining standard compliance.

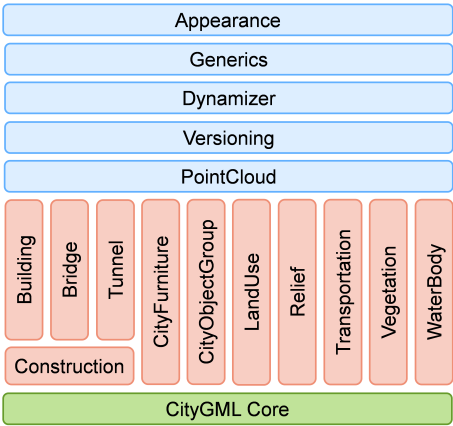


Figure 3.1: Overview of CityGML 3.0 modules showing the Core module, thematic extension modules, and additional extension modules. Source: (?)

3.2.2 CityJSON

CityJSON is a JSON-based (?) encoding format that implements a subset of the CityGML conceptual model (?). It is an official **ogc!** community standard (?) currently at version 2.0.1, supporting CityGML 3.0.0. While both CityGML and CityJSON implement the CityGML conceptual model, CityJSON exhibits several notable differences.

The following properties of CityJSON are derived from ?:

Flattened City Objects Architecture

CityJSON implements a flattened architecture where each city object receives a unique identifier, contrasting with CityGML’s hierarchical structure. While CityGML maintains a hierarchical organisation, CityJSON stores all objects at the same level (e.g., first and second-level city objects are stored in the same dictionary). To preserve hierarchical relationships, CityJSON uses a **parents** field to reference each object’s parent.

Geometry

CityJSON supports the same 3D geometric primitives as CityGML. However, instead of storing vertex coordinates directly within geometric primitives, CityJSON maintains a separate **vertices** array containing all coordinates. Geometric primitives then reference vertex positions within this array.

Semantic Surfaces

CityJSON stores semantic surfaces as separate objects, recognising that city objects often share common semantics. This is implemented through **semanticSurfaces** fields and a **values** array that maps surfaces to their corresponding semantic surface objects.

This is an example of how the semantic surfaces look like (derived from ?):

```

1  {
2    "type": "Solid",
3    "lod": 2,
4    "boundaries": [
5      [[0,3,2,1,22]], [[4,5,6,7]], [[0,1,5,4]],
6      [[1,2,6,5]] ]
7    ],
8    "semanticSurfaces": {
9      "surfaces" : [
10       { "type": "RoofSurface" },
11       {
12         "type": "WallSurface",
13         "paint": "blue"
14       },
15       { "type": "GroundSurface" }
16     ],
17     "values": [ [0, 1, 1, 2] ]
18   }
19 }
20 }
```

Geometry Templates

CityJSON implements CityGML's Implicit Geometry concept through "geometry templates". The format includes **geometry-templates** fields with a **templates** array that stores reusable geometries. City objects utilising these templates specify "GeometryInstance" in their geometry's **type** field to indicate template reuse.

This code shows an example of a geometry template derived from ?:

```

1  {
2    "geometry-templates": {
3      "templates": [{
4        "type": "MultiSurface",
5        "lod": 2,
6        "boundaries": [
7          [[0, 3, 2, 1]],
8          [[4, 5, 6, 7]],
9          [[0, 1, 5, 4]]
10       ]
11      }],
12     "vertices-templates": [...]
13   }
14 }
```

And this is how a city object references this template:

```

1  {
2    "type": "SolitaryVegetationObject",
3    "geometry": [
4      {
```

3 Related Work

```
5         "type": "GeometryInstance",
6         "template": 0,
7         "boundaries": [372],
8         "transformationMatrix": [
9             2.0, 0.0, 0.0, 0.0,
10            0.0, 2.0, 0.0, 0.0,
11            0.0, 0.0, 2.0, 0.0,
12            0.0, 0.0, 0.0, 1.0
13        ]
14    }
15 ]
16 }
```

Coordinate Quantisation

CityJSON employs coordinate quantisation to reduce geometry size. The **transform** field contains **scale** and **translate** values for coordinate quantisation. The original coordinates are recovered using the following formula (e.g., for the x component of vertex v):

$$x = v_x \cdot \text{transform.scale}_x + \text{transform.translate}_x \quad (3.1)$$

This is an example of how the transform object looks like (derived from ?):

```
1 {
2   "transform": {
3     "scale": [0.01, 0.01, 0.01],
4     "translate": [4424648.79, 5482614.69, 310.19]
5   }
6 }
```

Extension Mechanism

CityJSON implements an extension mechanism using JSON Schema, similar to CityGML's **ade!** (**ade!**). ? provides an overview of the developments of **ade!** in CityGML. While CityJSON's extension mechanism maintains compatibility with the core CityGML conceptual model, it has some limitations compared to CityGML's **ade!**, particularly in terms of inheritance and namespace support. The JSON Schema defines the data structure of extensions and can be used to validate extended objects.

CityJSON supports four distinct ways to extend the data model:

- Adding new properties at the root level of a CityJSON object (property names must start with "+", e.g., "+census")
- Defining additional attributes for existing city objects (attribute names must start with "+", e.g., "+colour")
- Creating new semantic objects (object names must start with "+", e.g., "+ThermalSurface")
- Creating or extending new city object types (city object names must start with "+", e.g., "+NoiseBuilding")

Each extension must be documented and validated using a JSON Schema file. This schema file must contain specific properties that define the structure and constraints of the extension. For example, an extension schema might look like this (derived from ?):

```

1  {
2    "type": "CityJSON_Extension",
3    "name": "Noise",
4    "uri": "https://someurl.org/noise.json",
5    "version": "0.1",
6    "description": "Extension to model the noise"
7    "extraRootProperties": {},
8    "extraAttributes": {},
9    "extraCityObjects": {}
10 }
```

These characteristics, particularly JSON's absence of repetitive closing element tags and the implementation of coordinate quantisation, result in significantly smaller file sizes compared to CityGML, achieving compression factors of up to $7\times$ (?).

The proposed data format I developed in this research, FlatCityBuf, inherits key concepts from CityJSON including semantic surfaces, geometry templates, coordinate quantisation, and the extension mechanism. The strategy for encoding city objects in FlatCityBuf will be explained in ??.

3.2.3 CityJSON Text Sequences (CityJSONSeq)

? optimises CityJSON for streaming applications by decomposing objects into independent sequences. The fundamental unit of CityJSONSeq is the **CityJSONFeature**, which represents a single feature encompassing a complete city object and its hierarchical children. For instance, a **CityJSONFeature** representing a "Building" includes its associated "BuildingPart" and "BuildingInstallation" objects. Unlike standard CityJSON objects that share vertices and appearances across multiple features, each **CityJSONFeature** maintains local vertex lists and appearance data, ensuring complete self-containment of geometric and visual information. Throughout this research, **CityJSONFeature** objects are referred to simply as "features". ?? (sourced from ?) demonstrates how CityJSONSeq's vertices are stored as local vertices arrays per each feature, contrasting with the shared vertex approach of standard CityJSON.

CityJSONSeq adheres to the Newline Delimited JSON specification (?), implementing a structured file format with specific requirements. The first line of a CityJSONSeq file must contain a CityJSON object that stores commonly used data shared across all features, including coordinate transformation parameters (**transform**), format version information (**version**), metadata (**metadata**), reusable geometry templates (**geometry-templates**), and extension definitions (**extensions**). This initialisation object establishes the global context for subsequent features. The example below shows how a **CityJSONFeature** is represented (derived from ?):

```

1  {
2    "type": "CityJSONFeature",
3    "id": "id-1",
4    "CityObjects": {
5      "id-1": {
6        "type": "Building",
7        "attributes": {
8          "roofType": "gabled roof",
9          "children": ["my balcony"]

```

3 Related Work

```
10     },
11     "geometry": [ ... ]
12   },
13   "my balcony": {
14     "type": "BuildingInstallation",
15     "parents": ["id-1"],
16     "geometry": [ ... ]
17   }
18 },
19 "vertices": [ ... ],
20 "appearance": { ... }
21 }
```

figs/related_work_theoretical_bg/cj_idea.pdf

Figure 3.2: Comparison of CityJSON and CityJSONSeq showing how vertices are stored.
Source: (?)

While CityJSONSeq generally provides improved compression and memory efficiency compared to standard CityJSON, it may produce larger file sizes in scenarios where features share minimal vertex counts or extensively share vertices and textures, due to the necessity of localising previously shared resources.

Despite CityJSONSeq’s improvements for streaming applications, its text-based JSON format exhibits several limitations that impede optimal cloud-native performance. The format lacks explicit data typing, storing all values as text strings irrespective of their semantic type (integers, floating-point numbers, booleans), resulting in increased storage overhead and requiring additional parsing operations. Moreover, the JSON structure demands complete data parsing and copying during processing operations, thereby constraining memory efficiency. The absence of built-in indexing mechanisms further restricts efficient spatial and attribute-based querying capabilities. These limitations create opportunities for enhanced cloud-native optimisation through binary encoding schemes that preserve native data types and enable zero-copy access patterns, complemented by integrated indexing mechanisms for efficient data retrieval operations.

3.2.4 3DBAG API

3DBAG is both a dataset and a project dedicated to generating 3D building models for the entire Netherlands (?). This open data initiative leverages the Register of Buildings and Addresses (BAG) as building footprints and the Actueel Hoogtebestand Nederland (AHN) (?), which serves as the national height model (point cloud) of the Netherlands. The 3DBAG data is automatically generated through the integration of BAG and AHN datasets and is publicly available in multiple formats including CityJSON (?), Wavefront OBJ (?), GeoPackage (?), and FlatGeobuf (?).

The 3DBAG **api!** (**api!**) (?) represents one of the project’s key web services, providing programmatic access to the dataset by returning CityJSONFeature objects as responses.

3.2.5 Enhancements to CityJSON Performance

Binary Encoding of CityJSON

? conducted a systematic evaluation of binary encoding techniques for CityJSON. This was done to address challenges associated with transmitting large-scale 3D city models over the web. The study assessed various compression and encoding methodologies, including CBOR, zlib, Draco and their combinations. It evaluated visualisation time, querying time, spatial analysis time, editing time, file size compression and lossiness. The analysis determined that the combination of CBOR and zlib offers optimal general-purpose efficiency due to its implementation simplicity. Conversely, Draco exhibited superior performance for pre-compressed data scenarios. However, the study identified limitations in Draco’s applicability. Specifically, it noted the increased complexity and computational overhead when handling smaller datasets. While these findings provide valuable insights for binary encoding implementations, they do not address optimisations tailored to cloud-native environments, particularly partial data retrieval over the web without requiring complete file downloads.

Experimental Implementation Using FlatBuffers

? explored the application of FlatBuffers (?) for encoding CityJSONFeature. This was done to enhance performance in cloud-native environments. The preliminary implementation revealed potential advantages in several key areas:

3 Related Work

- Faster feature access time.
- Lower memory consumption.
- Decreased storage requirements.

Building upon Peters’ initial work, which focused solely on basic CityJSONFeature encoding, this research develops a comprehensive solution that incorporates essential capabilities including spatial indexing, attribute indexing, extensions, textures, and geometry templates. The implementation specifically targets cloud-native environments, prioritising both scalability and efficient data processing to address the limitations of the preliminary approach.

3.3 Non-Geospatial Formats in Cloud Environments

Modern cloud-optimised geospatial formats leverage established non-geospatial data structures. These enhance efficiency in data transfer, storage and processing operations. Notable implementations include GeoParquet (?), which employs Parquet (?) for optimised geospatial data management. ? is constructed on FlatBuffers (?). Mapbox Vector Tiles (?) utilise Protocol Buffers (Protobuf) (?). These underlying formats are meticulously designed to improve performance metrics. These include serialisation/deserialisation speed, memory utilisation and data compression.

3.3.1 FlatBuffers

FlatBuffers is a cross-platform serialisation library developed by ?. It is optimised for efficient data access and transfer. The detailed characteristics and technical implementation of FlatBuffers is explained in ??. Benchmark analyses (?) indicate that FlatBuffers outperforms alternative serialisation formats. These include Protobuf (?) and JSON, in terms of deserialisation efficiency and memory utilisation.

3.3.2 Protocol Buffers (Protobuf)

Protobuf, developed by ?, represents a binary serialisation framework. It employs schema-based encoding mechanisms for data serialisation. This framework implements similar fundamental operations to FlatBuffers. These include schema definition and binary encoding processes. Notable differences between Protobuf and FlatBuffers include several operational characteristics (?):

- **Memory Limitations:** Requires complete dataset loading into memory, thereby limiting its applicability for large-scale data processing tasks.
- **Data Parsing Requirements:** Unlike FlatBuffers’ zero-copy access, Protobuf requires data parsing during deserialisation, introducing additional computational overhead.
- **Mutability Constraints:** Protobuf allows data modification after deserialisation, whereas FlatBuffers maintains immutable data structures, affecting performance characteristics.
- **Language Support and Community:** Protobuf benefits from broader language support and more extensive community adoption compared to FlatBuffers.

3.3.3 Apache Parquet

Apache Parquet (?) is a columnar storage format designed to support high-performance compression and encoding schemes for managing extensive datasets. The Parquet ecosystem includes the specification for the Parquet format (?), and various libraries for encoding and decoding Parquet files.

Parquet employs the record shredding and assembly algorithm (?) to effectively flatten nested data structures. Additionally, it implements efficient compression and encoding schemes tailored to column-level data, enhancing both storage efficiency and query performance.

The format utilises a hierarchical data organisation structure consisting of multiple levels:

- **File:** The top-level container that includes metadata and may contain the actual data.
- **Row Group:** A logical horizontal partitioning of data into rows, with each row group containing one column chunk per column in the dataset.
- **Column Chunk:** A contiguous chunk of data for a particular column within a specific row group.
- **Page:** The smallest indivisible unit within column chunks, serving as the fundamental unit for compression and encoding operations.

?? shows the structure of the Parquet format. This hierarchical organisation enables efficient data access patterns and optimised compression strategies at different granularity levels.

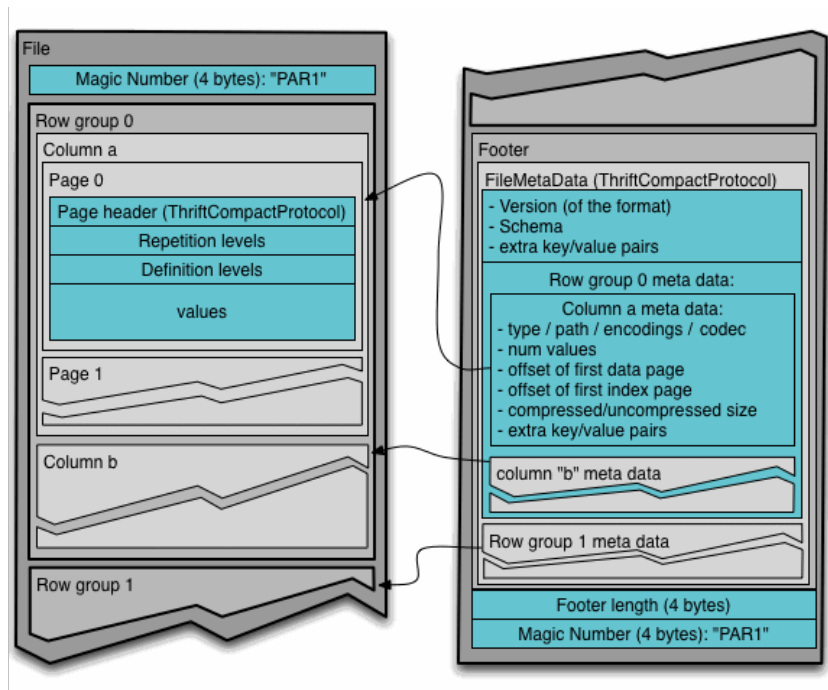


Figure 3.3: Parquet structure. Source: ?

3.3.4 Comparison of Non-Geospatial Formats

Existing research has evaluated the performance characteristics of non-geospatial formats within cloud environments. [?] conducted a comparative analysis of FlatBuffers and Protobuf. This focused on metrics such as serialisation/deserialisation efficiency, memory utilisation, and message size optimisation. Their investigation utilised randomised message sizes to assess format performance in vehicle-to-server communication scenarios. The analysis yielded the following observations:

- **Processing Efficiency:** Protobuf demonstrated superior serialisation performance but exhibited reduced deserialisation efficiency relative to FlatBuffers.
- **Memory Optimisation:** FlatBuffers consistently displayed lower memory consumption during both serialisation and deserialisation operations.
- **Data Compression:** Protobuf achieved greater message size reduction compared to FlatBuffers.

These findings advocate for the selection of FlatBuffers in applications where deserialisation performance and memory efficiency are paramount in data processing operations.

3.4 Cloud-Optimised Geospatial Implementations

Contemporary cloud-optimised geospatial implementations encompass formats such as Mapbox Vector Tiles ([?]), FlatGeobuf ([?]), PMTiles ([?]), and GeoParquet ([?]).

3.4.1 Mapbox Vector Tiles (MVT)

[?] implements a vector tile specification optimised for web-based data delivery. The format utilises Protobuf ([?]) for the serialisation of two-dimensional geospatial data and adopts a tile pyramid structure to enhance data retrieval operations.

3.4.2 PMTiles

PMTiles offers a standardised format for managing tile data addressed through Z/X/Y coordinates, supporting both vector and raster tile implementations. The format leverages **http! Range Requests** ([?]) to facilitate selective tile retrieval, thereby optimising network resource utilisation.

3.4.3 FlatGeobuf

FlatGeobuf adheres to the Simple Features [?] specification by utilising FlatBuffers ([?]) for serialisation. The architecture of FlatGeobuf enables efficient serialisation, deserialisation, and data processing operations through its packed Hilbert R-tree spatial index. This indexing mechanism, combined with stream search capabilities, allows clients to selectively retrieve and process specific geographic regions without necessitating the loading of the entire dataset.

Users and applications can effectively choose subsets of data based on spatial queries, optimising both storage and retrieval operations. ? provides a comprehensive guide for implementers of FlatGeobuf.

3.4.4 GeoParquet

GeoParquet integrates Parquet’s columnar storage architecture to facilitate optimised geospatial data operations (?). The format promotes interoperability across cloud data warehouse platforms, including BigQuery (?), Snowflake (?), and Redshift (?). Key technical characteristics of GeoParquet include:

- **Compression Efficiency:** Achieves superior compression ratios relative to alternative storage formats through columnar data organisation.
- **Optimised Read Operations:** The columnar architecture enables selective column access and efficient data filtering via predicate pushdown mechanisms, thereby enhancing performance in read-intensive workflows.

3.4.5 3D Tiles

3D Tiles, an **ogc!** standard (?), provides specifications for streaming and rendering extensive three-dimensional urban models. The format implements GLTF (?), a WebGL-optimised specification designed for efficient streaming in web environments.

The data structure employs spatial partitioning through bounding volumes, enabling selective rendering based on camera viewpoint requirements. While this architecture demonstrates optimal performance for visual rendering tasks, it presents limitations in two key areas: (1) arbitrary spatial extent retrieval and (2) attribute-based feature querying capabilities.

3.4.6 Comparative Analysis of Cloud-Optimised Geospatial Formats

While acknowledging the inherent limitations of direct format comparisons due to their distinct design objectives and application domains, ?? presents a systematic analysis of key operational characteristics across various cloud-optimised geospatial formats. The evaluation criteria and their corresponding scales are detailed in ??. This analysis facilitates the understanding of format-specific capabilities within their respective operational contexts.

This comparison provides a general overview but is inherently subjective. Direct comparisons may not be entirely fair as each format employs different optimisation strategies and targets different use cases. For example, comparing serialisation performance across formats with varying spatial indexing approaches may not yield meaningful insights, as the indexing overhead varies significantly between implementations.

Table 3.1: Comparative Analysis of Cloud-Optimised Geospatial Formats

Characteristics	FlatGeobuf	MVT	GeoParquet	GeoJSON	3D Tiles
Serialisation Performance					
Deserialisation Performance					 ^a
Storage Efficiency					
Memory Utilisation					
Implementation Complexity					
Spatial Indexing		 ^b	 ^c		 ^d
Random Access Support					

^a Optimised for GPU rendering
^b Tile-based partitioning
^c Random access to the internal chunks
^d Volumetric hierarchical partitioning

Table 3.2: Evaluation Criteria Color Legend

Color	Performance Level	Description
 Red	Poor	Very slow performance, high resource usage, or not supported
 Orange	Below Average	Slow performance or high complexity
 Yellow	Average	Moderate performance, compression, or basic support
 Green	Excellent	Very fast performance, high efficiency, or full support
 Gray	Not Applicable	Feature not available or not applicable to this format

3.5 Research Gaps

While existing studies have made advances in optimising CityJSON through various encoding techniques, there remains a deficiency in approaches specifically tailored for 3D city models in cloud environments. Several geospatial data formats have successfully implemented cloud-native optimisations (as discussed in ??).

Specifically, while advanced serialisation frameworks like FlatBuffers (detailed in ??) have proven effective in cloud-optimised geospatial formats, their application to 3D city models has not been thoroughly investigated. For example, FlatGeobuf for Simple Features (?) has shown success. This research endeavours to address this gap by systematically evaluating and implementing encoding methodologies. These methodologies aim to enhance decoding efficiency and query flexibility within cloud infrastructures, with the potential to achieve file size reduction through optimised binary encoding. The proposed approach is detailed in ??.

4 Methodology

This chapter presents the design and implementation of FlatCityBuf, a cloud-optimised binary format for 3D city models based on CityJSONSeq. The proposed approach addresses the limitations of existing formats through efficient binary encoding, spatial indexing, attribute indexing, and support for partial data retrieval.

4.1 Overview

4.1.1 Methodology Approach

Current 3D city model formats like CityGML, CityJSON, and **cjseq!** exhibit limitations in cloud environments with large-scale datasets, including retrieval latency, inefficient spatial querying without additional software support, and insufficient support for partial data access.

This chapter addresses these limitations through three interconnected objectives:

1. Development of a binary encoding strategy using FlatBuffers that preserves semantic richness while achieving faster read performance
2. Implementation of dual indexing mechanisms—spatial (packed Hilbert R-tree) and attribute-based (**s+tree!**) that accelerate query performance
3. Integration of cloud-native data access patterns through **http!** Range Requests, enabling partial data retrieval

4.1.2 Outcomes of the Methodology

Before delving into the methodological details, it is important to highlight the tangible research outcomes produced through this work:

- **Data format specification:** FlatCityBuf, a cloud-optimised binary format for 3D city models that maintains semantic compatibility with CityJSON while enabling efficient cloud-based access patterns.
- **Reference implementation:** A comprehensive Rust library for encoding, decoding, and querying FlatCityBuf files, accompanied by command-line interface (CLI) tools for conversion and validation.
- **Web demonstration:** A web-based prototype application that showcases the partial data retrieval capabilities of FlatCityBuf through **http!** range requests, demonstrating practical performance improvements in real-world scenarios.

- **Performance evaluation:** A comprehensive performance evaluation of the proposed methodology, demonstrating the benefits of the proposed approach in terms of file size, query latency, memory usage, and other relevant metrics.

These outcomes collectively address the research objectives by providing both a theoretical framework and practical implementations that validate the approach to cloud-optimised 3D city model storage and retrieval.

4.1.3 File Structure Overview

The FlatCityBuf format implements a structured binary encoding with five sequentially arranged components:

- **Magic bytes:** Eight-byte identifier ('F', 'C', 'B', '0', '1', '0', '0', '0') for format validation
- **Header section:** Contains metadata, attributes schema definitions, and CityJSON properties
- **Spatial index:** Implements a Packed Hilbert R-tree (?) for efficient geospatial queries
- **Attribute index:** Utilises a **s+tree!** for accelerated attribute-based filtering
- **Features section:** Stores features encoded as FlatBuffers tables



Figure 4.1: Physical layout of the FlatCityBuf file format, showing section boundaries and alignment considerations for optimised range requests

This sequence-based structure enables incremental file access through **http! Range Requests**—critical for cloud-based applications where minimising data transfer is essential.

4.1.4 Note on Binary Encoding

FlatCityBuf follows two key conventions for encoding binary data throughout the file format:

1. **Size-prefixed FlatBuffers:** All FlatBuffers records (header and features) include a 4-byte unsigned integer prefix indicating the buffer size. This enables programs to know the size of the record without parsing the entire content. The FlatBuffers **api!** implements this through `finish_size_prefixed` or equivalent language-specific methods.
2. **Little-endian encoding:** For data encoded outside FlatBuffers records (particularly in spatial and attribute indices), little-endian byte ordering is consistently applied, matching the endianness convention used within FlatBuffers records. This includes numeric values such as 32-bit and 64-bit integers, floating-point numbers, and offset values within indices.

These conventions ensure consistency across the file format and maximise compatibility with modern **cpu!** architectures, most of which use little-endian byte ordering. The size-prefixing mechanism is particularly important for cloud-based access patterns, as it facilitates precise **http!** Range Requests when retrieving specific file segments.

4.2 Magic Bytes

The magic bytes section comprises the first eight bytes of the file:

- The first three bytes contain the ASCII sequence 'FCB' for FlatCityBuf (0x46 0x43 0x42), serving as an immediate identifier
- The remaining five bytes represent the version number of the file format, compliant with Semantic Versioning (SemVer) (?). As the current version is 0.1.0, the magic bytes are 'FCB01000' (0x46 0x43 0x42 0x30 0x31 0x30 0x30 0x30). The last two bytes are reserved for future use and must be set to zero.

This signature design enables applications to validate file type and version compatibility without parsing the entire header content. The approach was directly inspired by FlatGeoBuf's methodology, which uses 'FGB' (F, G, B characters) in its magic bytes to indicate 'FlatGeoBuf' (?).

4.3 Header Section

The header section encapsulates metadata essential for interpreting the file contents, implemented as a size-prefixed FlatBuffers-serialised **Header** table. The header serves a dual purpose: it maintains compatibility with CityJSON by encoding the equivalent of the first line of a **cjseq!** stream (?)—which contains the root CityJSON object with metadata, coordinate reference system, and transformations—while adding FlatCityBuf-specific extensions for optimised retrieval and indexing. The full schema definition for the header can be found in ??.

In a **cjseq!** file, the first line contains a valid CityJSON object with empty **CityObjects** and **vertices** arrays but with essential global properties like **transform**, **metadata**, and **version**. The FlatCityBuf header encodes these same properties alongside additional indexing information required for cloud-optimised access patterns.

4.3.1 CityJSON Metadata Fields

Here are the core header fields with their data types and significance:

- **version** - *string (required)* - CityJSON version identifier (e.g., "2.0"), required field from CityJSON specification (?)
- **transform** - *Transform struct* - Contains scale and translation vectors enabling efficient storage of vertex coordinates through quantisation, derived from CityJSON's transform object (?)
- **reference_system** - *ReferenceSystem table* - **crs!** (**crs!**) information including:
 - **authority** - *string* - Authority name, typically "EPSG"
 - **code** - *string* - Numeric identifier of the **crs!**
 - **version** - *string* - Version of the **crs!** definition
- **geographical_extent** - *GeographicalExtent struct* - 3D bounding box containing min/-max coordinates for the dataset (?)

- **identifier** - *string* - Unique identifier for the dataset
- **title** - *string* - Human-readable title for the dataset
- **reference_date** - *string* - Date of reference for the dataset
- **point of contact** - *Contact table* - Contact information for the dataset provider (?), containing:
 - **poc_contact_name** - *string* - Name of the point of contact
 - **poc_contact_type** - *string* - Type of contact (e.g., "individual", "organisation")
 - **poc_role** - *string* - Role of the contact (e.g., "author", "custodian")
 - **poc_email** - *string* - Email address of the contact
 - **poc_website** - *string* - Website for the contact
 - **poc_phone** - *string* - Phone number of the contact
 - **poc_address_*** - *string* - Address components including thoroughfare number, name, locality, postcode, country

4.3.2 Appearance Information

Fields storing global appearance definitions:

- **appearance** - *Appearance table* - Container for visual representation properties, following CityJSON's appearance model (?), containing:
 - **materials** - *Array of Material tables* with properties defined in ??

Table 4.1: Material properties in the FlatCityBuf appearance model

Property	Data Type	Description
name	string	Required string identifier for the material
ambient_intensity	double	Double precision value from 0.0 to 1.0
diffuse_color	Array of double	Array of double values (RGB) from 0.0 to 1.0
emissive_color	Array of double	Array of double values (RGB) from 0.0 to 1.0
specular_color	Array of double	Array of double values (RGB) from 0.0 to 1.0
shininess	double	Double precision value from 0.0 to 1.0
transparency	double	Double precision value from 0.0 to 1.0
is_smooth	boolean	Boolean flag for smooth shading

- **textures** - *Array of Texture tables* with properties defined in ??
- **vertices_texture** - *Array of Vec2 structs* - Array containing UV coordinates (u,v), each coordinate value must be between 0.0 and 1.0 for proper texture mapping
- **default_theme_material** - *string* - String identifying default material theme for rendering when multiple themes are defined
- **default_theme_texture** - *string* - String identifying default texture theme for rendering when multiple themes are defined

Table 4.2: Texture properties in the FlatCityBuf appearance model

Property	Data Type	Description
type	TextureFormat enum	TextureFormat enum (PNG, JPG)
image	string	Required string containing image file name or URL
wrap_mode	WrapMode enum	WrapMode enum (None, Wrap, Mirror, Clamp, Border)
texture_type	TextureType enum	TextureType enum (Unknown, Specific, Typical)
border_color	Array of double	Array of double values (RGBA) from 0.0 to 1.0

The appearance model standardises visual properties of city objects, with materials defining surface properties and textures mapping images onto geometry. This separation from geometry allows efficient storage through shared material and texture references.

4.3.3 Geometry Templates

Fields supporting geometry reuse:

- **templates** - *Array of Geometry tables* - Reusable geometry definitions that can be instantiated multiple times, following CityJSON's template concept (?)
- **templates_vertices** - *Array of DoubleVertex structs* - Double-precision vertices used by templates, stored separately from feature vertices for higher precision in the local coordinate system (?)

The templates mechanism enables significant storage efficiency for datasets containing repetitive structures such as standardised building designs, street furniture, or vegetation. The detailed structure of geometry encoding, including boundary representation and semantic surface classification, will be explained further in ??.

4.3.4 Extension Support

Fields enabling to accommodate CityJSON's extension mechanism:

- **extensions** - *Array of Extension tables* - Definitions for CityJSON extensions (?), each containing:
 - **name** - *string* - Extension identifier (e.g., "+Noise")
 - **url** - *string* - Reference to the extension schema
 - **version** - *string* - Extension version identifier
 - **extra_attributes** - *string* - Stringified JSON schema for extension attributes
 - **extra_city_objects** - *string* - Stringified JSON schema for extension city objects
 - **extra_root_properties** - *string* - Stringified JSON schema for extension root properties
 - **extra_semantic_surfaces** - *string* - Stringified JSON schema for extension semantic surfaces

Unlike standard CityJSON (?), which references external schema definition files for extensions, FlatCityBuf embeds the complete extension schemas directly within the file as stringified JSON. This approach creates a self-contained, all-in-one data format that can be interpreted correctly without requiring access to external resources.

The embedding of extension schemas follows FlatCityBuf's design principle of maintaining file independence while preserving full compatibility with the CityJSON extension mechanism. The specific implementation details of how extended city objects and semantic surfaces are encoded in individual features will be explained further in ??.

4.3.5 Attribute Schema and Indexing Metadata

Fields supporting attribute interpretation and efficient querying:

- **columns** - *Array of Column tables* - Schema definitions for attribute data. This metadata is used to interpret the values of the attributes in the features. Each containing:
 - **index** - *int* - Numeric identifier of the column
 - **name** - *string* - Name of the attribute (e.g., "cityname", "owner", etc.)
 - **type** - *DataType enum* - Data type enumeration (e.g., "Int", "String", etc.)
 - **nullable** - *boolean* - Optional metadata for validating and interpreting values
 - **unique** - *boolean* - Optional metadata for validating and interpreting values
 - **precision** - *int* - Optional metadata for validating and interpreting values
- **semantic_columns** - *Array of Column tables* - Schema definitions for semantic surface attributes. Similar to the **columns** field, but specifically for interpreting attribute data attached to semantic surfaces in the geometry. This separation allows for different attribute schemas between city objects and their semantic surfaces.
- **features_count** - *ulong* - Total number of features in the dataset, enables client applications to pre-allocate resources
- **index_node_size** - *ushort* - Number of entries per node in the spatial index, defaults to 16, tuned for typical **http!** request sizes
- **attribute_index** - *Array of AttributeIndex structs* - Metadata for each attribute index, containing:
 - **index** - *int* - Reference to the column being indexed
 - **length** - *ulong* - Size of the index in bytes
 - **branching_factor** - *ushort* - Branching factor of the index, number of items in each node is equal to branching factor – 1
 - **num_unique_items** - *ulong* - Count of unique values for this attribute

The attribute schema system in FlatCityBuf is designed to efficiently interpret binary-encoded attribute values. The Column table structure is directly adopted from FlatGeoBuf's approach (?), which provides a flexible and extensible way to define attribute schemas. While optional fields such as **nullable**, **unique**, and **precision** are currently not utilized, they are included in the schema to accommodate potential future use cases.

4.3.6 Implementation Considerations

The header is designed to be compact while providing all necessary information to interpret the file. The size-prefixed FlatBuffers encoding enables efficient skipping of the header when only specific features are needed, important for cloud-based access patterns where minimising data transfer is essential. All numeric values in the header use little-endian encoding for consistency with modern architectures.

4.4 Spatial Indexing

Efficient spatial querying is a critical requirement for 3D city model formats, particularly in cloud environments where minimising data transfer is essential. FlatCityBuf implements a packed Hilbert R-tree spatial indexing mechanism (?) to enable selective retrieval of city features based on their geographic location. This section details the implementation approach, design decisions, and performance characteristics of the spatial indexing component.

4.4.1 The packed Hilbert R-tree

The spatial indexing mechanism implemented in FlatCityBuf directly adapts the packed Hilbert R-tree approach developed for FlatGeoBuf (?). The design combines several key features:

- A Hilbert curve-based spatial ordering strategy, inspired by Vladimir Agafonkin's flatbush library, which optimises data locality for spatially proximate features
- A "packed" R-tree implementation, where the tree is maximally filled with no empty internal slots, optimised for static datasets
- A bottom-up tree construction methodology that builds the index from pre-sorted features
- A flattened tree storage format that enables efficient streaming and remote access

The implementation details, including the Hilbert curve encoding algorithm and tree construction process, were sourced from FlatGeoBuf's reference implementation (?). Also, FlatGeoBuf's implementation is also inspired by Vladimir Agafonkin's flatbush library written in JavaScript (?). The Hilbert curve encoding algorithm, which converts 2D coordinates into a 1D space-filling curve, is based on a non-recursive algorithm described in ?. This approach, known as "2D-C" in spatial indexing literature (?), ensures that features with high spatial locality also have high storage locality, optimising io! operations for both local and remote access patterns.

The spatial indexing system is designed to support cloud-native access patterns, allowing efficient retrieval of data directly from cloud storage without requiring a persistent server process. This is achieved through a combination of the Hilbert-sorted feature ordering and the packed R-tree structure, which enables piecemeal access to both the index and feature data over http! requests.

It is important to explicitly acknowledge that the spatial indexing code in FlatCityBuf is a direct adaptation of FlatGeoBuf's implementation, with modifications primarily focused on integration with the 3D city model data structure rather than fundamental algorithmic changes. The original implementation by Björn Harrtell and other FlatGeoBuf contributors (?) provided an excellent foundation that has been proven effective for cloud-optimised geospatial data.

While the original FlatGeoBuf implementation targets 2D vector geometries, FlatCityBuf extends this approach to work with 3D city models by using two distinct spatial representations: the 2D bounding box centroid for Hilbert curve ordering, and the full 2D bounding box for spatial intersection tests. This dual representation allows efficient spatial indexing while maintaining accurate spatial query results. The decision to reuse this proven approach rather than

developing a novel indexing mechanism was based on FlatGeoBuf's demonstrated effectiveness for cloud-optimised geospatial data formats.

4.4.2 Feature sorting

A key optimisation in FlatCityBuf's indexing strategy is the spatial ordering of features using a Hilbert space-filling curve. This technique enhances data locality by ensuring that features which are spatially proximate in 2D (and 3D) space are also stored close together in the file, thereby optimising both disk access patterns and **http!** range requests (?).

The Hilbert curve encoding process for FlatCityBuf follows these steps:

1. For each feature, determine its 2D footprint by calculating the axis-aligned bounding box (minimum and maximum X,Y coordinates) from all vertices across all contained CityObjects
2. Calculate the geometric centroid of this 2D footprint
3. Apply a 32-bit Hilbert encoding algorithm to this centroid, converting the 2D spatial position into a 1D ordering value
4. Sort all features according to their computed Hilbert values in ascending order
5. During serialisation of the sorted features, record both the 2D bounding box and the byte offset (relative position from the start of the feature section) for each feature

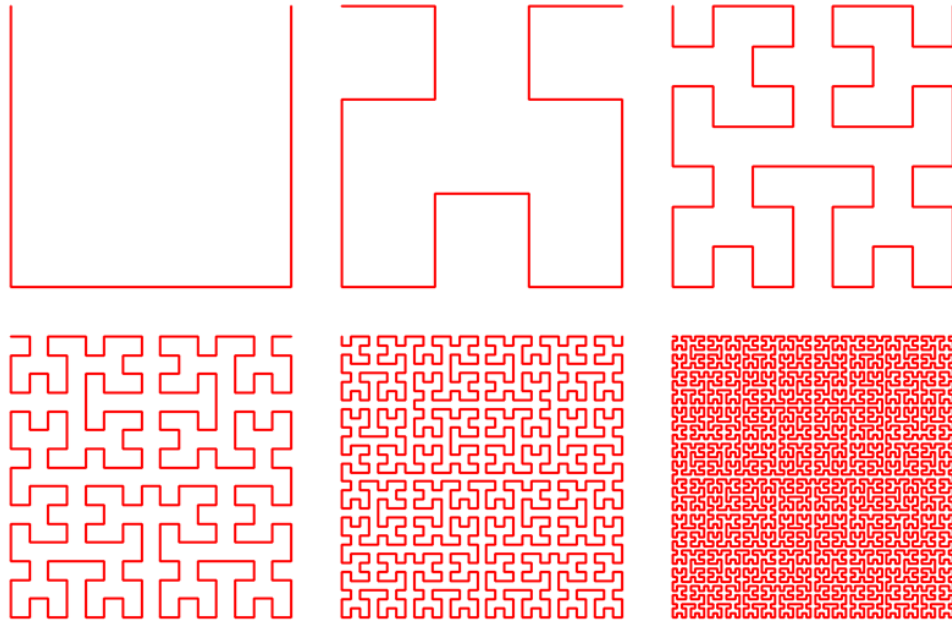


Figure 4.2: Example of a Hilbert curve. Image sourced from ?.

These recorded bounding boxes and byte offsets become the foundation for constructing the bottom layer of the R-tree index. The Hilbert encoding implementation uses a non-recursive

algorithm described in [?], which has been adapted from the FlatGeoBuf implementation ([?]), which in turn was inspired by a public domain implementation by [?].

This approach differs from traditional R-tree construction where nodes are built based on spatial proximity alone. By pre-sorting features along a space-filling curve before constructing the R-tree, FlatCityBuf achieves more predictable and efficient I/O patterns when performing spatial queries ([?]).

4.4.3 Index structure

The spatial index in FlatCityBuf is implemented as a packed Hilbert R-tree, with a flattened, level-ordered storage structure optimised for efficient traversal over **http!** range requests ([?]). The index is built bottom-up from the sorted features, creating a hierarchical structure where each node represents a spatial region containing its children.

Each index node in the spatial index is represented by a fixed-size binary structure containing:

- **Bounding box coordinates:** 4 little-endian double values (4 bytes each) defining the minimum and maximum X,Y coordinates of the node's bounding box
- **Byte offset:** A 64-byte unsigned integer pointing to either:
 - For leaf nodes: The position of the corresponding feature in the features section
 - For interior nodes: The position of the node's first child in the index section

This results in a fixed node size, allowing for predictable memory layouts and efficient search within each node level.

The tree is built using the following process:

1. Create the bottom layer (leaf nodes) using the bounding boxes and byte offsets recorded during feature serialisation
2. Group these leaf nodes according to their Hilbert-sorted order into parent nodes, with each parent node containing up to `index_node_size` children (configurable)
3. Compute the bounding box of each parent node as the union of its children's bounding boxes
4. Continue building upward, level by level, until a single root node is reached
5. Serialise the entire tree in level order, starting with the root, then its children, and so on, following the Eytzinger layout pattern (described in [??]) to optimise cache line utilisation during tree traversal

This "packed" structure ensures that the R-tree is maximally filled (except potentially for the rightmost nodes at each level), which is possible because the tree is built in bulk from a known static dataset. The total size of the index is deterministic and based solely on the number of features and the chosen node size.

Unlike traditional R-trees which support dynamic updates, the packed R-tree in FlatCityBuf is immutable after creation. This trade-off prioritises read performance and structural efficiency over the ability to modify the dataset, aligning with the file format's primary use case as a cloud-optimised geospatial data delivery mechanism ([?]).

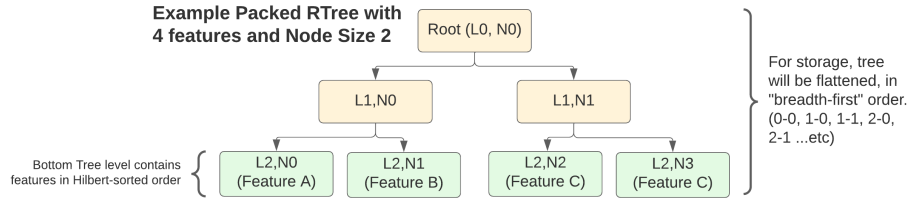


Figure 4.3: Example of a packed R-tree structure. Image sourced from ?.

4.4.4 2D vs 3D Indexing Considerations

Although FlatCityBuf is designed for 3D city models, the spatial indexing mechanism deliberately uses a 2D approach rather than a full 3D implementation. This design decision was based on several key observations:

- **Horizontal Distribution:** Most 3D city models are primarily distributed horizontally in global scale, with limited vertical extent relative to their horizontal footprint
- **Query Patterns:** Typical spatial queries for city models focus on horizontal regions (e.g., retrieving buildings within a district), rather than volumetric queries
- **Standards Compatibility:** OGC API - Features - Part 1: Core (?) and similar standards primarily support 2D spatial querying
- **Implementation Efficiency:** 2D indexing is computationally simpler and more storage-efficient than 3D alternatives

Conceptually, the generalization to 3D indexing is possible and would be beneficial for vertically dense metropolitan areas such as Tokyo or New York, where high-rise buildings create vertical distribution of features. In such environments, 3D spatial indexing could improve query performance for volumetric queries that consider height constraints or multi-level urban analysis. However, for the majority of urban environments and current use cases, the 2D approach seems to be sufficient and provides an optimal balance between implementation complexity and query performance.

4.5 Attribute Indexing

Attribute indexing is a fundamental component of the FlatCityBuf format, enabling efficient filtering and retrieval of city objects based on their non-spatial properties. This section details the requirements, design considerations, and implementation of the attribute indexing system.

4.5.1 Query Requirements Analysis

The attribute indexing system in FlatCityBuf was designed to support query patterns commonly encountered in geospatial applications. The prioritization of query operators was determined through analysis of established standards in the geospatial domain and common usage patterns in existing GIS software.

Common Query Operators in Geospatial Standards

Two major **ogc!** standards provide guidance on common query operators: Filter Encoding Standard (?) and Common Query Language (?). These standards define operators in several categories, as summarized in Table ??.

Table 4.3: Common query operators in geospatial standards

Category	OGC Filter Encoding	OGC CQL
Logical Operators	AND, OR, NOT	AND, OR, NOT
Comparison Operators	PropertyIsEqualTo, PropertyIsNotEqualTo, PropertyIsLessThan, PropertyIsGreaterThan, PropertyIsLessThanOrEqualTo, PropertyIsGreaterThanOrEqualTo, PropertyIsLike, PropertyIsNull, PropertyIsBetween	=, !=, <, <=, >, >=, LIKE, IS NULL, BETWEEN, IN
Spatial Operators	BBOX, Equals, Disjoint, Touches, Within, Overlaps, Crosses, Intersects, Contains, DWithin, Beyond	INTERSECTS, EQUALS, DISJOINT, TOUCHES, WITHIN, OVERLAPS, CROSSES, CONTAINS
Temporal Operators	After, Before, Begins, BegunBy, During, TContains, TEquals, TOverlaps, Ends, EndedBy, Meets, MetBy, OverlappedBy, AnyInteracts	AFTER, BEFORE, BEGINS, BEGUNBY, DURING, TCONTAINS, TEQUALS, TOVERLAPS, ENDS, ENDEDBY, MEETS, METBY, OVERLAPPEDBY, ANYINTERACTS
Additional Capabilities	ResourceId	Functions, Arithmetic Expressions, Array Operators

Priority Operators for FlatCityBuf

Based on this analysis and the practical constraints of optimising for cloud-based access, FlatCityBuf prioritises support for the following operators:

1. **Primary Comparison Operators:** Operators with direct index support
 - Equality (=)
 - Inequality (!=)
 - Less than (<)
 - Less than or equal (<=)
 - Greater than (>)
 - Greater than or equal (>=)
 - BETWEEN (implemented as combined $\geq$ and $\leq$)
2. **Logical Combinations:** Supported at the query execution level
 - AND (intersection of result sets)
 - NOT (negation of result sets)

While typical SQL supports more various operators (e.g., LIKE operators), these were not prioritised in the initial implementation as they are not part of either OGC Filter Encoding or OGC CQL standards, or because they require more complex index structures that are less commonly used in typical 3D city model queries.

By focusing on these high-priority operators, FlatCityBuf’s attribute indexing system aims to support the most common query patterns while maintaining efficient performance for cloud-based access. This approach provides capabilities that exceed current offerings such as the 3DBAG API, which primarily supports feature retrieval by identification attribute (`identification`) and is still working toward full **ogc!** compliance (?).

4.5.2 s+tree! Design and Modifications

After evaluating alternatives, a **s+tree!** with significant modifications was adopted for FlatCityBuf’s attribute indexing. **s+tree!** is a variant of the Static B+Tree that is specialised for read-only access patterns. Its theoretical background is described in ???. This decision was based on the following considerations:

- **io! Efficiency and Balanced Performance:** B+trees organise data into fixed-size nodes matching common **cpu!** cache sizes, offering $O(\log_B n)$ search complexity where B is the branching factor. This significantly reduces both the number of **io!** operations and network roundtrips compared to binary search, making it ideal for **http!** Range Requests where each roundtrip incurs substantial latency.

- **Query Versatility:** Unlike specialised data structures such as hash tables (optimised for exact matches) or sorted arrays (better for range queries), the B+tree structure provides a balanced performance across both exact match and range queries. While not achieving the optimal performance of specialised structures for specific query types, this balanced approach makes it well-suited for the diverse query patterns common in 3D city model applications.

s+tree! Characteristics

A s+tree! differs from a traditional B+tree in several important aspects:

- **Immutability:** Once constructed, the tree structure remains fixed, eliminating the need for complex rebalancing operations.
- **Perfect Node Fill:** All nodes except possibly the rightmost nodes at each level are filled to capacity, maximising space efficiency.
- **Predictable Structure:** The tree shape is determined solely by the number of elements and the node size, making navigation more efficient.
- **Bulk Construction:** The tree is built bottom-up in a single pass from sorted data, rather than through incremental insertions.

The original S+tree algorithm as described by ? provides an excellent foundation for read-only indexing. However, several significant modifications were necessary to adapt it to the specific requirements of FlatCityBuf:

- **Duplicate Key Handling:** 3D city model attributes often contain numerous duplicate values (e.g., hundreds of features with "Delft" as the value for "city name"). The s+tree! implementation described in literature (?) does not address the case of having duplicate values. The modified implementation incorporates a dedicated payload section that efficiently stores multiple feature references for identical attribute values without compromising the tree structure or search performance.

For handling duplicate keys in indexing structures, ? outlines three main approaches: (1) including duplicate entries in the index, (2) using variable-length records with a repeating pointer field, or (3) keeping fixed-length index entries with a single entry per key value and an extra level of indirection to handle multiple pointers. FlatCityBuf adopts the third approach, which is "more commonly used" according to ?, by implementing a payload section that stores the collection of feature offsets for each duplicate key. This design choice was made to maintain a simple implementation for search algorithms while efficiently handling attributes with potentially high duplicate cardinality. The fixed-length entries in the tree structure preserve the binary search efficiency, while the separate payload section accommodates the variable number of references without complicating the tree traversal logic.

- **Multi-type Support:** The index structure was extended to handle various attribute data types commonly found in 3D city models, including numeric types (integers, floating-point), string values, boolean flags, and temporal data (dates, timestamps).

- **Explicit Node Offsets:** While the original S+tree uses mathematical calculations to determine node positions, FlatCityBuf's implementation stores explicit byte offsets to child nodes. This modification simplifies the implementation without compromising performance. The parent node item has a 64-bit offset to the first child item of left child node.
- **Payload Pointer Mechanism:** To efficiently handle duplicate keys, the implementation uses a tag bit in the offset value to distinguish between direct feature references and pointers to the payload section. When the most significant bit is set, the remaining bits encode an offset to the payload section where multiple feature offsets are stored consecutively. This approach minimises both the storage overhead and redundant **http!** requests for unique keys while enabling support for duplicate keys.

These modifications ensure that the S+tree implementation is optimised for the specific characteristics of 3D city model data while preserving the performance advantages of the original algorithm.

4.5.3 Attribute Index Implementation

The attribute indexing system in FlatCityBuf is implemented as a binary encoded structure with four main components:

1. **Index Metadata:** Contains metadata about the index, including the column being indexed, branching factor, and number of unique values. This is stored in the header section of the file ??.
2. **Tree Structure:** A hierarchical arrangement of nodes with keys and pointers. Though it's called as "tree", it's conceptual structure. The actual structure is a linear sequence of nodes. Both internal and leaf nodes are stored consecutively in the "flat" structure.
3. **Payload Section:** Stores arrays of feature offsets for duplicate key values. Each payload entry has a 32-bit length prefix that indicates the number of feature offsets that follow.

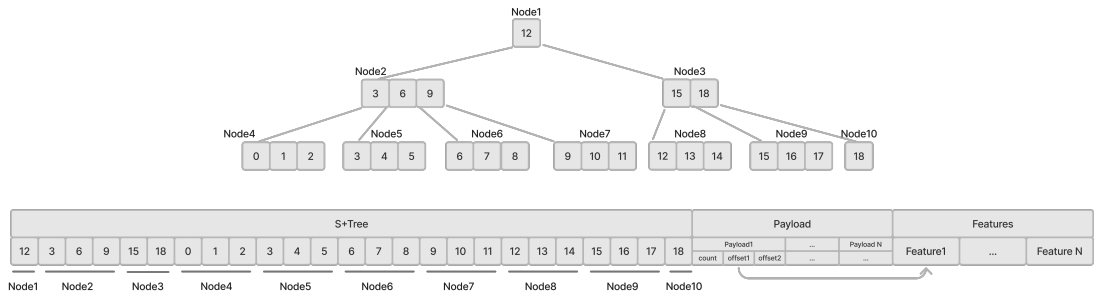


Figure 4.4: Attribute index implementation in FlatCityBuf

4.5.4 Construction of the Attribute Index

The construction of the attribute index follows these processes:

1. Create pairs of attribute values and their corresponding feature offsets.

2. Sort the pairs by the attribute values.
3. Create the payload section by grouping the feature offsets for duplicate attribute values.
4. Build the tree structure from the bottom layer up, using the configured branching factor and number of unique values to determine the tree's height and the array ranges for each level.
 - Leaf nodes: branching factor – 1 items are grouped together as one leaf node. Each item has a key and a u64 offset to either the feature or payload section.
 - Internal nodes: branching factor – 1 items are grouped together as one internal node. The key value of an internal node is the minimum key of the subtree of its right child node.
5. Structure the tree from bottom to top and write it to the file in the order from top to bottom.

4.5.5 Serialisation of Keys in the Tree

Key serialisation in the attribute index is a critical aspect of the implementation, directly affecting both storage efficiency and query performance. FlatCityBuf implements type-specific serialisation strategies that balance storage requirements, comparison efficiency, and implementation complexity.

Fixed-Length Value Serialisation

Fixed-length values offer significant advantages for tree structures, enabling predictable node sizes and efficient binary search within nodes. FlatCityBuf serialises fixed-length values using the following strategies:

- **Integer Types:** Primitive integer types (i8, i16, i32, i64, u8, u16, u32, u64) are serialised directly in their native binary format using little-endian byte order. For example, a u64 value occupies exactly 8 bytes in the index structure.
- **Floating-Point Types:** IEEE 754 floating-point values (?) use their native binary representation, with special handling for NaN values to ensure consistent ordering semantics. Since Rust's native floating-point types do not implement ordering traits, the implementation uses the `OrderedFloat` wrapper type from the Rust ecosystem to provide total ordering for floating-point values while preserving their binary efficiency.
- **Temporal Types:** Date and timestamp values are serialised using a normalised representation that preserves chronological ordering. Timestamps are encoded as a composite of two components: an i64 representing seconds since the epoch, followed by a u32 representing nanosecond precision, both in little-endian order. This 12-byte representation supports the full range of ISO 8601 datetime values with timezone information (?).
- **Boolean Values:** Boolean values are encoded as a single byte (0 for false, 1 for true), aligning with common binary encodings while ensuring consistent sort order. Since boolean attributes can only have two possible values (true or false), this results in a degenerate tree structure with just a single root node containing both values. While this implementation maintains consistency with the general indexing approach, boolean

attributes are rarely used as index keys in real-world 3D city model applications due to their limited discriminative power.

This direct serialisation approach for fixed-length types minimises both computational overhead during tree traversal and storage requirements in the index structure.

Variable-Length Value Serialisation

Supporting variable-length keys in B+tree structures presents significant implementation challenges. As ? notes, variable-length keys can lead to unpredictable node sizes and uneven fan-out, complicating both the tree construction and traversal algorithms. This issue is particularly relevant for string attributes in 3D city models, where key lengths can vary substantially.

Modern database systems typically address this challenge through techniques such as prefix compression, where only the distinguishing prefix of each key is stored in non-leaf nodes. For example, when indexing last names, a non-leaf node might store only "Nac" and "Nay" as the discriminating prefixes between "Nachamkin" and "Nayuddin" (?).

While implementing a full prefix compression scheme would be ideal, it would significantly increase the complexity of both the indexing algorithm and the format specification. After evaluating the trade-offs between implementation complexity and the practical requirements of 3D city model attribute data, FlatCityBuf adopts a pragmatic approach using fixed-length strings with a maximum length of 50 bytes. This length was selected based on analysis of common attribute values in 3D city datasets, where typical string attributes such as identifiers ("NL.IMBAG.Pand.0363100012345678"), city names ("Delft"), building types ("residential"), and similar values rarely exceed this length.

For strings shorter than the fixed length, padding with space characters ensures consistent key sizes throughout the tree structure. This approach simplifies implementation while still supporting the most common use cases found in 3D city model datasets. The space overhead from padding is generally acceptable given the relative infrequency of string attributes compared to numeric attributes in typical datasets.

4.5.6 Query Strategies

The attribute index implementation provides two core functions that enable efficient query execution:

- **find_exact_match**: Traverses the tree structure to locate an exact match for a specified key value.
- **find_partition_point**: Identifies the boundary positions within the tree for a given query value, essential for range-based operations.

These fundamental functions support both exact match and range queries. Range queries are implemented by determining lower and upper bounds using **find_partition_point** and then retrieving all results within those boundaries. For inequality queries, the implementation uses **find_exact_match** to identify the target item and then returns all items except the matched one. This query functionality aligns with the standard operators defined in ?:

- `PropertyIsEqualTo`

- `PropertyIsNotEqualTo`
- `PropertyIsLessThan`
- `PropertyIsGreaterThan`
- `PropertyIsLessThanOrEqualTo`
- `PropertyIsGreaterThanOrEqualTo`
- `PropertyIsBetween`

For complex logical operations, the implementation supports compound queries by executing multiple index lookups and combining the results. For **AND** operations, it computes the intersection of result sets, while **OR** operations would use the union of results. Currently, only the **AND** logical operator is fully implemented.

4.5.7 Streaming s+tree! over http!

The index is structured to optimize for **http!** Range Requests, with several techniques employed to minimize network overhead:

- **Streaming search:** The search algorithm operates in a streaming fashion, requesting only the nodes necessary for query evaluation in sequential order. This approach ensures that even with large indices, the system avoids loading the entire tree structure into memory, significantly reducing resource requirements.
- **Payload Prefetching:** Proactively caches parts of the payload section during initial query execution, reducing **http!** requests for duplicate keys.
- **Batch Payload Resolution:** Collects multiple payload references during tree traversal and resolves them with consolidated **http!** requests.
- **Request Batching:** Groups adjacent node requests to minimise network roundtrips.
- **Block Alignment:** Nodes are aligned to fixed size boundaries to match typical file system and **http!** caching patterns.

During query execution, the system interprets the provided condition (e.g., `building_height > 25`) and traverses the appropriate attribute index to find matching features. The search algorithm selects between `find_exact_match` for equality conditions or `find_partition_point` for range queries, adapting the traversal strategy accordingly. Results are returned as a set of feature offsets, which can then be used to retrieve the actual feature data from the features section of the file.

4.6 Feature Encoding

The feature encoding section of FlatCityBuf is responsible for the binary representation of 3D city objects and their associated data. This component preserves the semantic richness of the CityJSON model while leveraging FlatBuffers' efficient binary serialisation. The full schema definition for feature encoding can be found in ??.

4.6.1 CityJSONFeature and CityObject Structure

FlatCityBuf implements the core structure of **cjseq!** using the following FlatBuffers tables:

- **CityFeature** - *table (root object)* - The top-level container for city objects: The CityFeature properties are detailed in ??.

Table 4.4: CityFeature properties in the FlatCityBuf feature encoding

Property	Data Type	Description
id	string (key, required)	Required string identifier, marked as a key field for fast lookup
objects	Array of CityObject tables	Collection of individual 3D features
vertices	Array of Vertex structs	Quantised X,Y,Z coordinates (int32)
appearance	Appearance table	Optional visual styling information

- **CityObject** - *table* - Individual 3D city objects with properties detailed in ??

Table 4.5: CityObject properties in the FlatCityBuf feature encoding

Property	Data Type	Description
type	CityObjectType enum	Object classification (Building, Bridge, etc.) following CityJSON types
id	string (key, required)	Required string identifier, marked as a key field
geographical_extent	GeographicalExtent struct	3D bounding box of the object
geometry	Array of Geometry tables	Shape information
attributes	ubyte array	Binary blob containing attribute values (interpretable via columns schema)
columns	Array of Column tables	Schema defining attribute types and names
children	Array of string	IDs referencing child objects
children_roles	Array of string	Descriptions of relationship roles
parents	Array of string	IDs referencing parent objects
extension_type	string	Optional type for extended objects (e.g., "+NoiseBuilding")

This structure maintains CityJSON's hierarchical organisation while taking advantage of FlatBuffers' binary encoding and zero-copy access capabilities, with the exception of attributes which are self-encoded as binary blobs.

4.6.2 Geometry Encoding

Geometry in FlatCityBuf follows CityJSON's boundary representation (B-rep) model with flattened arrays for FlatBuffers encoding:

- **Geometry** - *table* - Container for geometric representation with properties detailed in ??

Table 4.6: Geometry properties in the FlatCityBuf feature encoding

Property	Data Type	Description
type	GeometryType enum	Geometric dimension type (0D-Point, 1D-LineString, etc.)
lod	float	Level of Detail value
boundaries	Array of uint32	Indices referencing vertices
strings	Array of uint32	Counts defining vertex groups
surfaces	Array of uint32	Counts defining string groups
shells	Array of uint32	Counts defining surface groups
solids	Array of uint32	Counts defining shell groups
semantics_boundaries	Array of uint32	Parallel arrays to boundaries for semantic classification
semantics_values	Array of SemanticObject tables	Semantic information for surfaces

- **SemanticObject** - *table* - Semantic classification of geometry parts with properties detailed in ??

Table 4.7: SemanticObject properties in the FlatCityBuf feature encoding

Property	Data Type	Description
type	SemanticSurfaceType enum	Surface classification (WallSurface, RoofSurface, etc.)
extension_type	string	Optional extended semantic type name
attributes	ubyte array	Binary blob containing semantic-specific attributes
columns	Array of Column tables	Schema defining attribute types and names
parent	uint32	Index to parent semantic object
children	Array of uint32	Indices to child semantic objects

- **GeometryInstance** - *table* - Reference to template geometry with properties detailed in ??

Table 4.8: GeometryInstance properties in the FlatCityBuf feature encoding

Property	Data Type	Description
transformation	TransformationMatrix struct	4×4 transformation matrix
template	uint32	Index referencing a template in the header section
boundaries	Array of uint32	Single-element array containing reference point index

- **Vertex** - *struct* - Quantised 3D coordinates with properties detailed in ??

Table 4.9: Vertex properties in the FlatCityBuf feature encoding

Property	Data Type	Description
x	int32	X coordinate, converted using header transform
y	int32	Y coordinate, converted using header transform
z	int32	Z coordinate, converted using header transform

Hierarchical Boundaries as Flattened Arrays

A key challenge in adapting CityJSON’s recursive boundary representation to FlatBuffers is that FlatBuffers does not support nested arrays. FlatCityBuf addresses this by implementing a dimensional hierarchy encoded as parallel flattened arrays:

The encoding strategy follows a dimensional hierarchy from lowest to highest dimension:

1. **boundaries:** A single flattened array of integer vertex indices
2. **strings:** Array where each value indicates the number of vertices in each ring/boundary
3. **surfaces:** Array where each value indicates the number of strings/rings in each surface
4. **shells:** Array where each value indicates the number of surfaces in each shell
5. **solids:** Array where each value indicates the number of shells in each solid

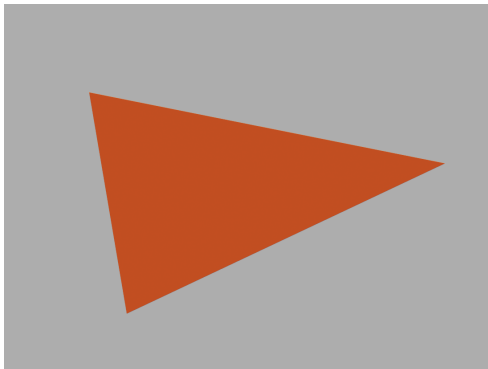


Figure 4.5: Example of a triangle encoded as a hierarchical boundary.

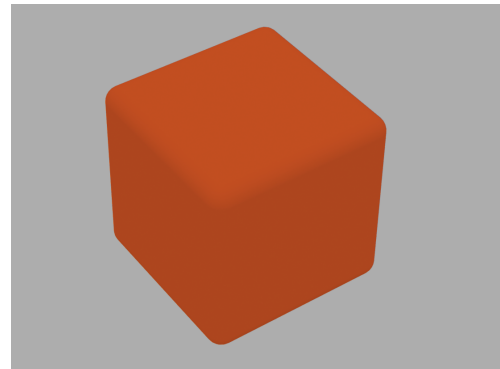


Figure 4.6: Example of a cube encoded as a hierarchical boundary.

For example, a simple triangle would be encoded as:

```

1 boundaries: [0, 1, 2]           // Indices of three vertices
2 strings: [3]                   // Single string with 3 vertices
3 surfaces: [1]                   // Single surface containing 1 string

```

Listing 4.1: Hierarchical boundary encoding for a simple triangle

A more complex structure such as a cube (a solid with 6 quadrilateral faces) would be encoded as:

```

1 boundaries: [0, 1, 2, 3, 0, 3, 7, 4, 1, 5, 6, 2, 4, 7, 6, 5, 0, 4, 5, 1,
    2, 6, 7, 3]
2 strings: [4, 4, 4, 4, 4, 4] // 6 strings with 4 vertices each
3 surfaces: [1, 1, 1, 1, 1, 1] // 6 surfaces with 1 string each
4 shells: [6] // 1 shell with 6 surfaces
5 solids: [1] // 1 solid with 1 shell

```

Listing 4.2: Hierarchical boundary encoding for a cube structure

Semantic Surface Encoding

Semantic surface information is encoded using a similar approach:

- **semantics_values:** Array of *SemanticObject* tables containing type classifications, attributes, and hierarchical relationships
- **semantics_boundaries:** Array of indices that reference entries in *semantics_values*, with a parallel structure to the geometry boundaries

This parallel structure allows each geometric component to have associated semantic information without requiring deeply nested structures. For example, in a building model where each face has a semantic classification (wall, roof, etc.), the *semantics_boundaries* array would have the same structure as the *boundaries* array, with each surface having a corresponding semantic value.

Through this flattened array approach, FlatCityBuf preserves the rich hierarchical structure of CityJSON geometries while conforming to FlatBuffers' efficiency-oriented constraints on data organisation.

Geometry Template Encoding

FlatCityBuf implements CityJSON's template mechanism for efficient representation of repeated geometry patterns, a common requirement in urban environments where many buildings, street furniture items, vegetation, or other objects share identical geometric structures. The template approach separates the geometry definition from its instantiation:

- **Template Definition:** Templates are defined once in the header section as full Geometry objects with characteristics detailed in ??

Table 4.10: Template Definition characteristics in FlatCityBuf geometry templates

Characteristic	Description
Geometry format	Templates use the same Geometry table format described previously for standard geometries
Vertex precision	Template vertices are stored with double-precision coordinates (DoubleVertex) to maintain accuracy in the local coordinate system
Vertex storage	All template vertices for all templates are stored in a single flat array (templates_vertices)
Index referencing	Indices within template boundaries reference positions in this dedicated template vertex array

- **Template Instantiation:** CityObjects reference templates through GeometryInstance tables with properties detailed in ??

Table 4.11: Template Instantiation properties in FlatCityBuf geometry templates

Property	Data Type	Description
template	uint	A single unsigned integer index referencing a specific template in the header
boundaries	Array of uint	Contains exactly one index referencing a vertex in the feature's vertex array, which serves as the reference point for placement
transformation	TransformationMatrix	A 4×4 transformation matrix (rotation, translation, scaling) that positions the template relative to the reference point

FlatCityBuf preserves CityJSON's template mechanism, which provides significant storage efficiency by storing repeated geometries once and referencing them with transformation parameters.

4.6.3 Materials and Textures

FlatCityBuf supports CityJSON's appearance model through the following structures:

- **Appearance** - *table* - Container for visual styling information with properties detailed in ??

Table 4.12: Appearance properties in FlatCityBuf feature encoding

Property	Data Type	Description
materials	Array of Material tables	Surface visual properties definitions
textures	Array of Texture tables	Image mapping information
vertices_texture	Array of Vec2 structs	UV coordinates for texture mapping
material_mapping	Array of MaterialMapping tables	Links materials to surfaces
texture_mapping	Array of TextureMapping tables	Links textures to surfaces
default_theme_material	string	Default material theme identifier
default_theme_texture	string	Default texture theme identifier

- **Material** - *table* - Surface visual properties with properties detailed in ??
- **Texture** - *table* - Image mapping information with properties detailed in ??
- **MaterialMapping** - *table* - Links materials to surfaces with properties detailed in ??
- **TextureMapping** - *table* - Links textures to surfaces with properties detailed in ??

This implementation prioritizes efficient storage by referencing external texture files rather than embedding image data directly, enabling selective loading based on application requirements while maintaining full compatibility with CityJSON's appearance model.

Table 4.13: Material properties in FlatCityBuf feature encoding

Property	Data Type	Description
name	string (required)	Unique material identifier
ambient_intensity	double	Value from 0.0 to 1.0
diffuse_color	Array of double	RGB values from 0.0 to 1.0
emissive_color	Array of double	RGB values from 0.0 to 1.0
specular_color	Array of double	RGB values from 0.0 to 1.0
shininess	double	Value from 0.0 to 128.0
transparency	double	Value from 0.0 to 1.0
is_smooth	boolean	Flag for smooth shading

Table 4.14: Texture properties in FlatCityBuf feature encoding

Property	Data Type	Description
type	TextureFormat enum	Format type (PNG, JPG)
image	string (required)	Image file name or URL
wrap_mode	WrapMode enum	Wrapping option (None, Wrap, Mirror, Clamp, Border)
texture_type	TextureType enum	Type classification (Unknown, Specific, Typical)
border_color	Array of double	RGBA values from 0.0 to 1.0

Texture Storage Design Rationale

FlatCityBuf stores texture references rather than embedding texture data directly for several strategic reasons:

- **Performance Priority:** Enables rapid loading of geometric and semantic data without the overhead of large texture files when not required.
- **On-demand Loading:** Supports selective texture loading based on application needs, beneficial for analysis-focused use cases.
- **Size Management:** Maintains reasonable file sizes for large-scale datasets.
- **Web Efficiency:** Individual texture files can be cached by browsers or **cdn!** (**cdn!**)s, significantly improving performance for repeated access in web applications.

This approach follows established patterns in formats like glTF, OBJ, and I3S, prioritizing operational efficiency over self-contained packaging for city-scale datasets.

4.6.4 Attribute Encoding

Attributes in FlatCityBuf are encoded as binary data with a schema defined through *Column* tables, which were detailed previously in ???. Rather than repeating column structure information, this section focuses on the binary encoding strategy:

- **Attribute Binary Encoding** - Efficient type-specific serialisation:
 - *Numeric types* - Native binary representation (little-endian)
 - *String* - Length-prefixed UTF-8 encoding

Table 4.15: MaterialMapping properties in FlatCityBuf feature encoding

Property	Data Type	Description
theme	string	Theme identifier (e.g., "summer", "winter")
values	Array of uint32	Indices to surfaces or boundaries
material	uint32	Index to the referenced material

Table 4.16: TextureMapping properties in FlatCityBuf feature encoding

Property	Data Type	Description
theme	string	Theme identifier (e.g., "summer", "winter")
values	Array of uint32	Indices to surfaces or boundaries
texture	uint32	Index to the referenced texture
uv_indexes	Array of uint32	Indices to UV coordinates

- *Boolean* - Single byte (0 = false, 1 = true)
- *Date/DateTime* - Standardised binary format
- *Byte array* - Length-prefixed binary data
- *Nested JSON* - Length-prefixed JSON string encoding of complex nested structures
- *Null* - Not encoded to save space (null attributes are omitted from the binary representation)

FlatCityBuf encodes attributes as type-specific binary values with a corresponding schema definition. Each attribute is stored as a key-value pair where the key is the column index and the value is the binary representation of the attribute. This approach balances flexibility with reasonable performance while maintaining compatibility with the original CityJSON semantic model. The figure below illustrates how different attribute types are encoded in the binary format.

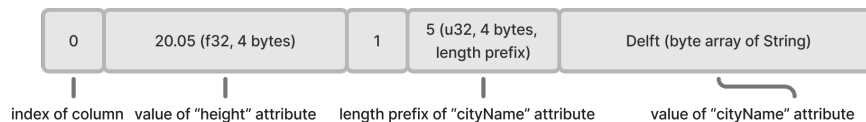


Figure 4.7: Example of attribute encoding in FlatCityBuf

4.6.5 Extension Mechanism

FlatCityBuf provides comprehensive support for CityJSON's Extension mechanism, which was previously detailed in ???. While the Extension structures are defined in the header, their implementation within actual city features requires specific encoding strategies that balance extensibility with performance.

Encoding of Extended City Objects

Extended city object types (those prefixed with "+") are encoded using a two-part strategy:

- A standard enum value `ExtensionObject` is used for the `type` field to distinguish whether the extended object or core object
- The actual extension type name (e.g., "+NoiseCityFurnitureSegment") is stored in the `extension_type` string field (This will be null for core objects)

Encoding of Extended Semantic Surfaces

Similarly, extended semantic surface types follow the same pattern:

- The `type` field uses the enum value `ExtraSemanticSurface`
- The specific type (e.g., "+ThermalSurface") is stored in the `extension_type` field (This will be null for core objects)

Extension Attribute Encoding

Extension-specific attributes are encoded using the same binary serialisation mechanism as core attributes:

- Extension attributes are included in the same binary representation as standard attributes
- The schema for these attributes is stored alongside the `columns` of the `Header` table (See ??)

During decoding:

- The same decoding logic is applied as for core attributes
- If needed, the application can identify extension attributes by checking if the column name begins with "+"

Adding New Properties at the Root of a Document

Adding new properties at the root level follows the same principle as adding extension attributes to existing City Objects. Since root-level extensions are also arbitrary structured JSON objects, fields starting with "+" are treated as extension attributes. The attribute encoding mechanism does not distinguish whether attribute keys start with "+" or not, so no special treatment is required.

However, since each City Object can already have extended attributes, this approach is rarely needed in practice. Consequently, we have not yet implemented this functionality.

Unlike CityJSON, which references external schema files for extensions, FlatCityBuf's self-contained approach ensures that all extension information is available within a single file. This approach maintains the cloud-optimised philosophy of minimising external dependencies while preserving full compatibility with the rich extension capabilities of CityJSON.

4.7 HTTP Range Requests and Cloud Optimisation

A critical component of cloud-optimised geospatial formats is their ability to support selective data retrieval without downloading entire datasets. FlatCityBuf achieves this capability through strategic implementation of HTTP Range Requests (?), enabling efficient partial data retrieval. This section details the technical implementation, optimisation strategies, and cross-platform compatibility of this mechanism.

4.7.1 Principles of Partial Data Retrieval

http! Range Requests, defined in RFC 7233 (?), allow clients to request specific byte ranges from server resources instead of entire files. This capability is fundamental to FlatCityBuf's cloud-optimised design. Since each feature in FlatCityBuf is length-prefixed, once the client knows the byte offset to a specific feature, can request precisely the bytes needed. While data access patterns vary—from sequential access to spatially or attribute-indexed retrieval—the core principle remains consistent: fetch only the necessary data.

4.7.2 Range Request Workflow

The **http!** Range Request workflow in FlatCityBuf follows a carefully optimised sequential process:

1. **Header Retrieval:** The client first requests the magic bytes (8 bytes) and **Header** (described in ??). This initial request provides essential metadata including coordinate reference systems, transformations, the total number of features, and index structure information etc..
2. **Index Navigation:** Based on query parameters (spatial bounding box or attribute conditions), the client selectively navigates the appropriate index structures:
 - For spatial queries, the client traverses only the relevant nodes of the packed Hilbert R-tree along the query path
 - For attribute queries, the client similarly traverses only the necessary portions of the appropriate **s+tree!** indices
3. **Feature Resolution:** Using byte offsets obtained from the indices, the client makes targeted range requests for specific features. The size of each feature is determined implicitly by the difference between consecutive offsets. The absolute byte offset of a feature within the file can be calculated by summing the size of the **Magic bytes**, the size of the **Header**, the size of the **indices**, and the relative offset of the feature.
4. **Progressive Processing:** Features are processed incrementally as they arrive, allowing applications to begin rendering or analysis before all data is received, significantly improving perceived performance.

This workflow enables efficient partial data retrieval by leveraging indexing strategies to minimize both the number of HTTP requests and the total data volume transferred.



Figure 4.8: **http!** Range Request workflow in FlatCityBuf showing the sequential process of header retrieval, index navigation, and selective feature retrieval. The client makes targeted requests for specific byte ranges rather than downloading the entire dataset.

4.7.3 Optimisation Techniques

Network latency often dominates performance when accessing data over **http!**, with each request incurring significant overhead regardless of payload size. FlatCityBuf implements several techniques to minimise this overhead:

- **Request Batching:** Multiple feature requests are grouped into larger, consolidated HTTP requests rather than making individual requests for each feature. This approach significantly reduces the number of HTTP round trips, improving overall performance while minimising network overhead.
- **Payload Prefetching:** As explained in ??, when an attribute index is about to be used, the implementation proactively downloads a portion of its payload section. This anticipatory approach reduces latency for subsequent operations by having relevant data already available in memory when needed.
- **Streaming Process of Indices:** Both spatial and attribute indices implement a streaming approach where only the necessary node items in the tree structure are loaded when needed. Rather than loading entire index structures upfront, the system traverses the tree on demand, requesting only the relevant portions required for the current query.
- **Buffered http! Client:** The implementation leverages a buffered **http!** client library developed by ? that efficiently caches previously fetched data ranges, avoiding redundant requests when overlapping ranges are accessed.

These optimisations work in concert to minimise the number of **http!** requests, resulting in significantly improved performance for cloud-based 3D city model applications.

5 Result

5.1 Overview

This chapter presents comprehensive evaluations of the FlatCityBuf format, demonstrating its performance characteristics and practical applicability through multiple assessment approaches. The evaluation encompasses both technical performance metrics and real-world implementation scenarios to provide a holistic understanding of the format’s capabilities.

The chapter is structured around several key evaluation components. First, a web prototype implementation is presented to demonstrate FlatCityBuf’s practical capabilities in browser environments, showcasing how the format enables partial data access for large 3D city models through **http!** Range Requests.

Second, the datasets used throughout the evaluation are described, including both the established benchmark datasets from ? and additional PLATEAU datasets from the Takeshiba district of Tokyo, providing context for the performance comparisons and ensuring reproducibility of results.

Third, file size comparisons are conducted across different encoding formats to evaluate storage efficiency and compression characteristics of FlatCityBuf relative to existing CityJSON variants.

Finally, web environment evaluations demonstrate real-world performance characteristics by measuring data retrieval times and network efficiency in browser-based scenarios. These tests specifically evaluate the effectiveness of **http!** Range Requests for selective data access, providing insights into bandwidth optimisation and response times critical for web-based 3D city model applications.

The following sections present detailed results from each evaluation component, culminating in integrated analyses that synthesise findings to provide comprehensive insights into FlatCityBuf’s performance characteristics and practical benefits for 3D city model applications.

5.1.1 Web Prototype

To demonstrate FlatCityBuf’s capabilities in web environments and illustrate practical user interactions with the data, a functional web prototype was developed. The prototype is publicly accessible at <https://fcb-web-prototype.netlify.app/>. It leverages the WebAssembly module of FlatCityBuf combined with TypeScript and React for the frontend implementation, with Cesium serving as the 3D map rendering engine.

The prototype operates on a substantial dataset covering approximately 20km × 20km of South Holland, Netherlands, stored as a single 3.4GB FlatCityBuf file. This file is delivered directly from Google Cloud Storage(?), a serverless storage service, where it exists as a static file similar to images or videos, requiring no specialized server-side processing. Despite this large file size,

5 Result

the application remains responsive by utilising the **http!** range request capabilities described in ?? . Users can interact with the data through several query mechanisms:

- **Spatial queries:** Users can filter features either by defining a spatial bounding box or by placing a point on the map to retrieve features based on intersection or nearest-neighbor relationships.
- **Attribute queries:** The interface supports filtering features through attribute conditions (e.g., `building id = 1`, `height > 10m`), demonstrating the attribute index capabilities.
- **Data export:** Users can download the filtered subset of features in **cjseq!** format, showcasing the format conversion capabilities.

This prototype effectively demonstrates how FlatCityBuf enables browser-based applications to work with large 3D city models without downloading the entire dataset, providing responsive performance even on consumer-grade hardware and network connections.

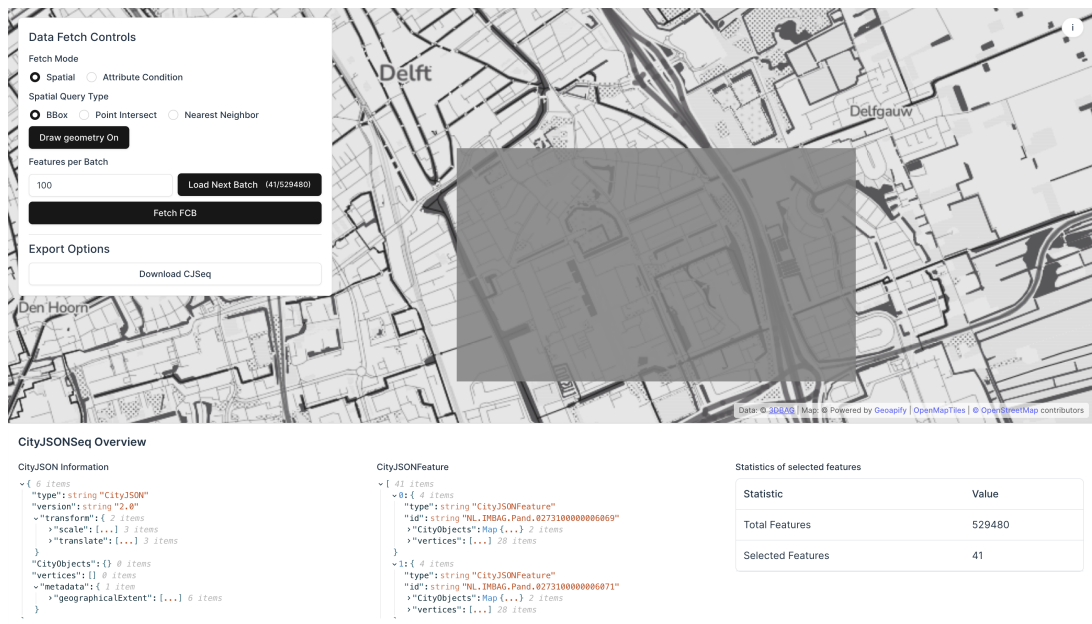


Figure 5.1: Web prototype of FlatCityBuf demonstrating spatial and attribute query capabilities on a 3.4GB dataset of South Holland.

5.1.2 Cross-Platform Implementation

FlatCityBuf provides range request capabilities across multiple platforms to maximise accessibility and integration options:

FlatCityBuf is implemented primarily as a Rust library that can be used in both native environments and web browsers. The same codebase is compiled to:

- Native Rust library for server-side applications and desktop GIS tools

- **wasm!** module for browser-based applications with JavaScript interoperability

This cross-platform approach enables FlatCityBuf to work with both Rust’s native **http!** clients and browser-based Fetch **api!** implementations. The **wasm!** implementation has one notable limitation: current browser **wasm!** implementations use a 32-bit memory model (4GB limit), which may constrain processing of country-level datasets. This limitation will be resolved with the upcoming **wasm!** Memory64 proposal (?).

5.1.3 Integration with Cloud Infrastructure

The **http!** Range Request mechanism integrates seamlessly with modern cloud infrastructure. FlatCityBuf files can be served from standard object storage services like AWS S3, Google Cloud Storage, or Azure Blob Storage, all of which support range requests without additional server-side processing. This enables a serverless architecture where the client-side filtering approach eliminates the need for dedicated server-side processing. This infrastructure compatibility ensures that FlatCityBuf can be deployed in cost-effective cloud environments without requiring specialised application servers and databases.

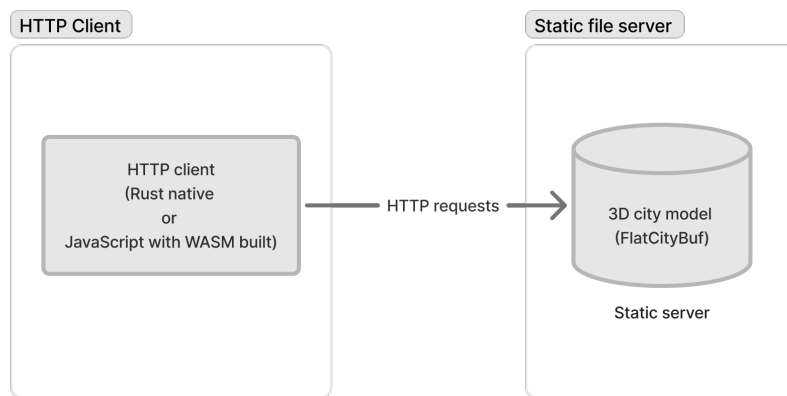


Figure 5.2: Server architecture for FlatCityBuf. The client-side filtering approach eliminates the need for dedicated server-side processing.

5.2 Datasets

To evaluate file sizes and conduct both local and web-based benchmarks, we employed a diverse range of datasets from ? supplemented with additional datasets from PLATEAU (?). PLATEAU employs its own data specification combining CityGML 2.0 (?) with a custom **ade!**. The CityGML-encoded 3D city models were converted to CityJSON using `citygml-tools`, a command-line utility developed by ?. Note that PLATEAU’s **ade!** components are not included in the converted datasets since they are not compatible with the current CityJSON implementation.

To comprehensively evaluate FlatCityBuf, we utilised diverse PLATEAU models including buildings, bridges, transport, tunnels, and vegetation. The dataset collection spans various

Table 5.1: The datasets used for the benchmark.

	dataset		size of file			attributes			
	CityObj	CityFeat app. ^(a)	CityJSONSeq	FlatCityBuf	compr. ^(b)	verts avg ^(c)	obj ^(d)	sem ^(e)	
3DBAG	2221	1110	6 MB	6 MB	−6%	82612	74	37	1
3DBV	71634	71634	317 MB	281 MB	12%	4992893	69.70	64	0
Helsinki	77267	77231	412 MB	345 MB	16%	3039107	39.35	27	9
Helsinki_{tex}	77267	77231 tex	644 MB	545 MB	15%	3039107	39.35	28	9
Ingolstadt	379	55	4 MB	3 MB	19%	88001	1600	33	13
Montréal	294	294 tex	5 MB	5 MB	−4%	32242	110	0	0
NYC	23777	23777	95 MB	76 MB	20%	1044145	44	3	3
Rotterdam	853	853 tex	3 MB	3 MB	−4%	26679	31	5	0
Vienna	1322	307	5 MB	4 MB	14%	47229	154	7	4
Zürich	198699	52834	247 MB	189 MB	24%	3564542	6.6	8	0
PLATEAU (Building)	10405	4307	77 MB	79 MB	−3%	147754	34	14	2
PLATEAU (Bridge)	60	8	5 MB	5 MB	−9%	16357	2045	5	2
PLATEAU (Railway)	412	412	4 MB	4 MB	−2%	5846	14	3	2
PLATEAU (Transport)	8136	8136	26 MB	26 MB	−1%	45992	5.65	3	2
PLATEAU (Tunnels)	21	3	5 MB	5 MB	4%	12306	4102	4	1
PLATEAU (Vegetation)	936	936	2 MB	2 MB	−31%	2567	2.74	3	0

^a appearance: ‘tex’ indicates textures are stored; ‘mat’ indicates materials are stored

^b compression factor is $\frac{\text{CityJSONSeq} - \text{FlatCityBuf}}{\text{CityJSONSeq}}$ (positive values indicate size reduction)

^c average number of vertices per feature

^d number of attributes in city objects

^e number of semantic surface attributes in city objects

urban environments from small-scale architectural models to large metropolitan areas across European and Japanese cities, enabling evaluation across different feature types, geometric complexities, and data modeling approaches.¹

5.3 File Size Comparison

5.3.1 File size results

?? presents a comparison of datasets in both **cjseq!** and FlatCityBuf formats. The results demonstrate that FlatCityBuf encoding achieves superior compression for several datasets, including Helsinki, Ingolstadt, and New York City, with compression factors of 16%, 19%, and 20% respectively. Conversely, the PLATEAU datasets exhibit the opposite trend, with **cjseq!** format demonstrating better storage efficiency.

5.3.2 Analysis of file size results

Although ?? provides a summary of file size comparisons, the factors influencing these outcomes require further investigation. This section analyses the underlying causes through controlled experiments with simplified datasets.

¹The datasets used in this evaluation are publicly available at <https://github.com/HideBa/flatcitybuf-data>.

Level of detail

To examine how **lod!** (**lod!**) affects file size, I conducted a series of tests using the TU Delft BK building model at various **lod!** levels. Each **lod!** variant was systematically extracted from the original model, with attributes and semantic information deliberately removed to isolate the effect of geometric complexity. ?? presents the results of this analysis.

Since each test dataset contains only a single city feature, we compare feature sizes rather than total file sizes. This approach is necessary because FlatCityBuf incorporates a larger header structure, which would disproportionately affect comparisons involving minimal features.

The results demonstrate that while file sizes increase with higher levels of detail, the compression factor remains largely independent of **lod!**. Both formats show similar proportional growth in size as geometric complexity increases, with FlatCityBuf consistently achieving approximately 25% size reduction compared to CityJSONSeq regardless of the **lod!** level. This suggests that the compression efficiency is determined by the underlying format design rather than the geometric complexity of the data.

Table 5.2: Comparison of file sizes across different levels of detail for the TU Delft BK building model.

Dataset	FlatCityBuf ^(a)	CityJSONSeq ^(b)	Compression	Vertices
TUD BK All	140 kB	189 kB	26%	4549
TUD BK lod! 0	13 kB	21 kB	38%	785
TUD BK lod! 1.2	37 kB	49 kB	25%	1350
TUD BK lod! 1.3	45 kB	60 kB	25%	1600
TUD BK lod! 2.2	62 kB	83 kB	25%	2168

Average feature size in bytes in FlatCityBuf: $\frac{\text{Total FlatCityBuf size}}{\text{Number of features}}$
 Average feature size in bytes in CityJSONSeq: $\frac{\text{Total CityJSONSeq size}}{\text{Number of features}}$

Attributes

To assess the impact of attributes on file size, we tested simple cube models from (?) with varying numbers of attributes. We systematically generated random attributes for each test case, examining both integer and string data types to determine their effect on compression efficiency. ?? presents the results of this analysis.

The randomly generated attributes in our test datasets followed a consistent pattern, as shown in the example below:

```

1 {
2   "type": "Building",
3   "geometry": [...],
4   "attributes": {
5     "attr_1": "value_1",
6     "attr_2": "value_2",
7     "attr_3": "value_3",
8     "attr_4": "value_4",
9     "attr_5": "value_5",
10    ...

```

5 Result

```

11   "attr_n": "value_n"
12   }
13 }

```

Listing 5.1: Example of a CityJSON feature with attributes

Table 5.3: Comparison of file sizes with varying numbers of attributes for simple cube models.

Dataset	FlatCityBuf ^(a)	CityJSONSeq ^(b)	Compression
10 attributes (int)	580 B	611 B	5%
100 attributes (int)	2 kB	2 kB	33%
1000 attributes (int)	12 kB	22 kB	44%
10 attributes (string)	580 B	611 B	5%
100 attributes (string)	2 kB	2 kB	33%
1000 attributes (string)	12 kB	22 kB	44%
Average feature size in bytes in FlatCityBuf: $\frac{\text{Total FlatCityBuf size}}{\text{Number of features}}$			
Average feature size in bytes in CityJSONSeq: $\frac{\text{Total CityJSONSeq size}}{\text{Number of features}}$			

For integer attribute tests, all values were randomly generated integers between 0 and 1000. For string attribute tests, values were randomly generated strings of varying lengths between 5 and 15 characters. This approach ensured a realistic representation of typical attribute data while maintaining controlled test conditions.

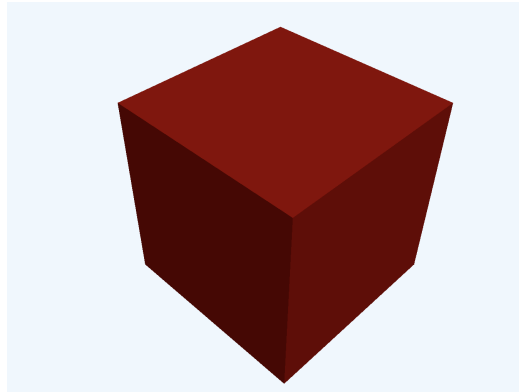


Figure 5.3: Simple cube model used for attribute testing. This basic geometric structure provides a controlled environment for evaluating the impact of attributes on file size.

The results reveal a clear pattern: FlatCityBuf’s compression advantage over CityJSONSeq increases substantially with the number of attributes. With only 10 attributes, the compression benefit is minimal at 5%, but rises markedly to 33% with 100 attributes and reaches 44% with 1000 attributes.

This efficiency stems from FlatCityBuf’s architectural design, which stores the attribute schema once in the file header. Each feature subsequently references attributes using only a 2-byte (u16) index, while CityJSONSeq must replicate identical attribute keys across all features. Although additional attributes increase the header size, this overhead is distributed across all features in the dataset. The header remains relatively compact—even with 1000 attributes, it occupies only a few tens of kilobytes.

These characteristics render FlatCityBuf particularly advantageous for datasets containing numerous attributes. The same efficiency applies to semantic surface attributes, where the schema-based approach provides similar compression benefits when features contain multiple surfaces with rich semantic information.

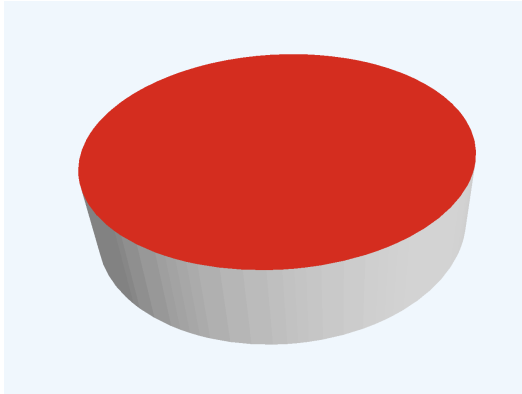
Geometry complexity

To evaluate how geometric complexity influences file size, we analysed models with varying numbers of vertices. The test utilised two geometrically distinct models from the TU Delft campus dataset—one simple and one complex. To isolate the effect of geometry, attributes and semantic information were removed, leaving only the essential geometric components required by CityJSON. ?? presents the numerical results of this analysis, while ?? provides visual comparisons of the models.

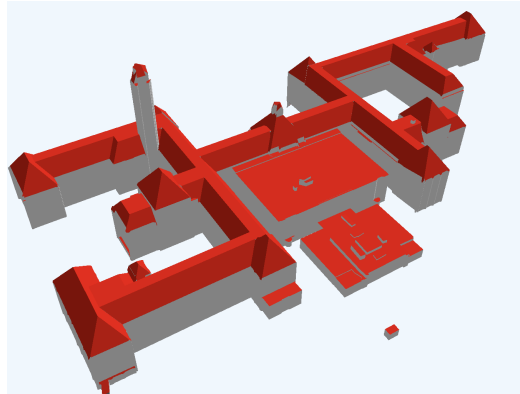
Table 5.4: Comparison of file sizes with varying geometric complexity.

Dataset	FlatCityBuf ^(a)	CityJSONSeq ^(b)	Compression	Vertices/Feature
TUD BK	140 kB	189 kB	26%	4549
TUD Simple	13 kB	15 kB	15%	340

Average feature size in bytes in FlatCityBuf: $\frac{\text{Total FlatCityBuf size}}{\text{Number of features}}$
Average feature size in bytes in CityJSONSeq: $\frac{\text{Total CityJSONSeq size}}{\text{Number of features}}$



(a) TUD Simple model (340 vertices/feature)



(b) TUD BK model (4549 vertices/feature)

Figure 5.4: Visual comparison of models with different geometric complexity.

The results demonstrate that geometric complexity significantly affects compression efficiency, with FlatCityBuf achieving better compression for more intricate models. The TU Delft BK building model, containing 4549 vertices per feature, exhibits a higher compression rate of 26% compared to the simpler model with 340 vertices at 15%.

This differential appears to result from the expanding boundary field as geometry becomes more complex. FlatCityBuf employs a strongly typed representation of boundaries (using u32 integers) that maintains a constant size for encoding each vertex, whereas CityJSONSeq requires additional bytes due to its text-based format. This fundamental difference in geometry encoding becomes increasingly advantageous for FlatCityBuf as geometric complexity rises.

Vertices and coordinates

To investigate how coordinate scale affects file size, I conducted tests using identical cube geometries with different coordinate magnitudes. All test models represent the same simple cube shape with identical geometric complexity (8 vertices per feature), but the coordinate values are scaled to different magnitudes—ranging from single digits to millions—while maintaining the same spatial relationships. ?? presents the results of this analysis, utilising the same base cube geometry as in ??.

Table 5.5: Comparison of file sizes with varying coordinate scales.

Dataset	FlatCityBuf ^(a)	CityJSONSeq ^(b)	Compression	Scale
Cube (1)	476 B	370 B	−29%	1
Cube (10)	476 B	459 B	−4%	10
Cube (1k)	476 B	507 B	6%	1,000
Cube (1M)	476 B	579 B	18%	1,000,000
Average feature size in bytes in FlatCityBuf: $\frac{\text{Total FlatCityBuf size}}{\text{Number of features}}$				
Average feature size in bytes in CityJSONSeq: $\frac{\text{Total CityJSONSeq size}}{\text{Number of features}}$				

The results reveal an intriguing relationship between coordinate scale and file size in both formats. FlatCityBuf maintains a consistent size of 476 bytes regardless of coordinate magnitude, demonstrating its fixed-size binary encoding for numeric values. In contrast, CityJSONSeq’s file size increases proportionally with larger coordinate values, growing from 370 bytes with single-digit coordinates to 579 bytes with million-scale coordinates.

This behaviour occurs because both FlatCityBuf and CityJSONSeq use integer values as coordinates, which are quantised by the **Transform** field as explained in ??. However, FlatCityBuf stores these coordinates as fixed-size 32-bit integers, while CityJSONSeq, being a text-based format, requires more characters to represent larger numbers. Consequently, FlatCityBuf transitions from being less efficient than CityJSONSeq for small coordinate values (−29%) to substantially more efficient for large coordinate values (18%).

This characteristic explains the pattern observed in ??. FlatCityBuf demonstrates lower storage efficiency for PLATEAU datasets, likely because these datasets employ geographic coordinate systems with values typically between −180 and 180. Since CityJSON quantises coordinates through the **Transform** field, latitude and longitude values can be represented as relatively small integers. Conversely, datasets where FlatCityBuf performs better—such as NYC and Helsinki—use local coordinate systems (in metres) with larger internal values, resulting in improved compression efficiency with FlatCityBuf.

Summary of File Size Analysis

The comprehensive analysis of various factors affecting file size reveals distinct patterns in the compression performance of FlatCityBuf compared to CityJSONSeq:

- **Level of Detail:** The analysis demonstrates that geometric detail levels have minimal impact on compression efficiency. While file sizes naturally increase with higher **lod**s, the compression advantage of FlatCityBuf remains relatively consistent at approximately 25% across different levels of geometric complexity.

- **Attribute Quantity:** The number of attributes significantly influences compression performance. FlatCityBuf's efficiency increases dramatically with attribute count, from minimal compression (5%) with 10 attributes to substantial compression (44%) with 1000 attributes. This progressive advantage stems from FlatCityBuf's schema-based approach that eliminates redundant attribute key storage.
- **Geometric Complexity:** More intricate geometries benefit from improved compression with FlatCityBuf. As boundary fields expand with geometric complexity, FlatCityBuf's fixed-size numeric representation provides greater efficiency compared to the text-based encoding of CityJSONSeq, increasing compression from 15% for simple geometries to 26% for complex models.
- **Coordinate Scale:** The magnitude of coordinate values has a significant impact on compression efficiency. FlatCityBuf's constant-size integer representation maintains consistent file sizes regardless of coordinate scale, while CityJSONSeq requires more space for larger values. This creates a transition from inferior compression (-29%) with small coordinate values to superior compression (18%) with large coordinate values.

These findings elucidate the observed variations in compression performance across different datasets in ???. FlatCityBuf demonstrates optimal performance for datasets with numerous attributes, complex geometries, and large-scale coordinate systems, while CityJSONSeq may retain advantages for simpler datasets with limited attributes and smaller coordinate values.

5.4 Benchmark on Local Environment

This section presents a comprehensive performance evaluation of the FlatCityBuf format conducted in a controlled local environment. The analysis focuses on critical metrics including read operations, memory utilisation, and processing efficiency to establish a thorough understanding of the format's performance characteristics.

5.4.1 Test Environment

All benchmarks were executed within a consistent hardware and software configuration to ensure reliability and reproducibility:

- **Hardware:** Apple MacBook Pro with M1 Max chip, 32GB unified memory
- **Operating System:** macOS Sequoia 15.4
- **Storage:** 1TB SSD with approximately 200GB available capacity
- **Runtime Environment:** Rust 1.86.0, with optimised release builds

5.4.2 Measurement Parameters

The benchmark framework captured multiple performance dimensions through the following key indicators:

- **Read Performance:** Time required to deserialise the file and map the data into memory using zero-copy techniques, measured in milliseconds with microsecond precision
- **Memory Efficiency:** Peak Resident Set Size (RSS) during file processing, providing an accurate measurement of maximum memory requirements

These parameters were systematically measured across all encoding formats—CityJSONSeq, CBOR, BSON, and FlatCityBuf—to facilitate direct performance comparisons. CBOR and BSON were selected as additional comparison formats because they are JSON-compatible binary encoding formats, providing a meaningful intermediate comparison between text-based CityJSONSeq and the custom FlatCityBuf implementation. For CBOR and BSON evaluation, single CityJSON files were encoded, which serve as the source for the corresponding CityJSONSeq datasets. Other data formats such as Protocol Buffers or GeoParquet would require developing dedicated libraries similar to the FlatBuffers implementation, making them less suitable for this comparative analysis. The subsequent sections present a detailed analysis of these measurements and their implications for practical applications.

5.4.3 Read Performance FlatCityBuf vs CityJSONSeq

The performance comparison between FlatCityBuf and CityJSONSeq was conducted across multiple datasets, measuring processing time, and memory consumption as key metrics. ?? presents these results.

Table 5.6: Performance comparison between CityJSONSeq and FlatCityBuf

Dataset	Processing Time			Memory Consumption		
	cjseq ^a	FlatCityBuf	Ratio ^b	cjseq ^a	FlatCityBuf	Ratio ^b
3DBAG	56.3 ms	6.6 ms	8.6×	23.9 MB	5.1 MB	4.7×
3DBV	3.99 s	122.5 ms	32.6×	283.8 MB	63.2 MB	4.5×
Helsinki	4.05 s	132.2 ms	30.6×	15.3 MB	5.2 MB	2.9×
Ingolstadt	37.2 ms	0.5 ms	75.8×	30.1 MB	6.9 MB	4.4×
Montréal	50.3 ms	0.6 ms	81.6×	36.3 MB	5.7 MB	6.4×
NYC	887.6 ms	42.9 ms	20.7×	20.6 MB	5.0 MB	4.1×
Rotterdam	22.2 ms	1.3 ms	17.6×	9.2 MB	4.4 MB	2.1×
Vienna	45.9 ms	1.9 ms	24.0×	14.6 MB	5.2 MB	2.8×
Zürich	1.88 s	151.9 ms	12.4×	31.3 MB	5.1 MB	6.2×
PLATEAU (Building)	861.4 ms	32.5 ms	26.5×	220.9 MB	64.4 MB	3.4×
PLATEAU (Bridge)	83.9 ms	0.3 ms	256.8×	75.0 MB	12.0 MB	6.3×
PLATEAU (Railway)	37.9 ms	2.0 ms	18.5×	19.0 MB	5.1 MB	3.8×
PLATEAU (Transport)	244.0 ms	13.3 ms	18.4×	76.7 MB	20.2 MB	3.8×
PLATEAU (Tunnels)	47.9 ms	1.9 ms	24.9×	70.6 MB	12.6 MB	5.6×
PLATEAU (Vegetation)	852.3 ms	32.9 ms	25.9×	189.8 MB	56.9 MB	3.3×

^a CityJSONSeq

^b Ratio = CityJSONSeq metric / FlatCityBuf metric (higher values indicate better FlatCityBuf performance)

The performance comparison reveals significant advantages for FlatCityBuf in both processing time and memory consumption. FlatCityBuf demonstrates consistently superior performance, processing data between 8.6×

datasets, which suggests that FlatCityBuf exhibits lower overhead when handling smaller datasets. Memory consumption is also consistently reduced compared to CityJSONSeq, with FlatCityBuf showing particularly notable advantages for certain datasets, including Montréal and the PLATEAU Bridge model.

5.4.4 Read performance FlatCityBuf vs CBOR

The performance comparison between FlatCityBuf and CBOR was conducted using the same datasets and measurement methodology. ?? presents these results.

Table 5.7: Performance comparison between CBOR and FlatCityBuf

Dataset	Processing Time			Memory Consumption		
	CBOR	FlatCityBuf	Ratio ^a	CBOR	FlatCityBuf	Ratio ^a
3DBAG	74.0 ms	6.6 ms	11.2×	194.1 MB	5.1 MB	38.1×
3DBV	6.34 s	122.5 ms	51.8×	4.96 GB	63.2 MB	80.3×
Helsinki	7.97 s	132.2 ms	60.3×	5.14 GB	5.2 MB	1011.2×
Ingolstadt	46.9 ms	0.5 ms	95.7×	187.5 MB	6.9 MB	27.3×
Montréal	58.4 ms	0.6 ms	94.7×	257.1 MB	5.7 MB	45.3×
NYC	1.33 s	42.9 ms	31.0×	1.65 GB	5.0 MB	337.4×
Rotterdam	30.8 ms	1.3 ms	24.4×	140.0 MB	4.4 MB	31.9×
Vienna	58.8 ms	1.9 ms	30.7×	179.8 MB	5.2 MB	34.7×
Zürich	3.53 s	151.9 ms	23.3×	4.51 GB	5.1 MB	913.2×
PLATEAU (Building)	1.06 s	32.5 ms	32.4×	1.83 GB	64.4 MB	28.4×
PLATEAU (Bridge)	63.6 ms	0.3 ms	194.7×	305.4 MB	12.0 MB	25.6×
PLATEAU (Railway)	46.3 ms	2.0 ms	22.7×	141.0 MB	5.1 MB	27.9×
PLATEAU (Transport)	316.1 ms	13.3 ms	23.8×	614.5 MB	20.2 MB	30.5×
PLATEAU (Tunnels)	147.7 ms	1.9 ms	76.7×	400.2 MB	12.6 MB	31.8×
PLATEAU (Vegetation)	997.8 ms	32.9 ms	30.3×	1.97 GB	56.9 MB	35.3×

^a Ratio = CBOR metric / FlatCityBuf metric (higher values indicate better FlatCityBuf performance)

The benchmark results demonstrate that FlatCityBuf consistently outperforms CBOR across all tested datasets. When compared to CBOR, FlatCityBuf achieved processing time improvements ranging from 11.2× to 194.7×, with particularly significant speedups observed for smaller datasets such as the PLATEAU bridge model and Ingolstadt. This pattern reflects FlatCityBuf’s zero-copy deserialization advantage, which provides proportionally greater benefits when parsing overhead dominates the total processing time in smaller datasets.

Memory consumption was reduced by factors ranging from 25.6× to 1011.2×, with the most dramatic improvements observed in larger datasets such as Helsinki and Zürich. This trend occurs because CBOR requires loading the entire dataset into memory during deserialization, while FlatCityBuf’s zero-copy approach allows selective access without full memory allocation. As dataset size increases, this fundamental difference in memory management strategy becomes increasingly pronounced.

5.4.5 Read performance FlatCityBuf vs BSON

The performance comparison between FlatCityBuf and BSON followed the same methodology as the previous comparisons. ?? presents the detailed results.

The benchmark results show that FlatCityBuf significantly outperforms BSON across all tested datasets. Processing time improvements ranged from 17.8× to 541.8×, with the most dramatic speedup observed for the PLATEAU bridge model. The exceptional performance gain for

Table 5.8: Performance comparison between BSON and FlatCityBuf

Dataset	Processing Time			Memory Consumption		
	BSON	FlatCityBuf	Ratio ^a	BSON	FlatCityBuf	Ratio ^a
3DBAG	117.1 ms	6.6 ms	17.8×	276.8 MB	5.1 MB	54.3×
3DBV	9.97 s	122.5 ms	81.4×	6.38 GB	63.2 MB	103.4×
Helsinki	14.64 s	132.2 ms	110.7×	6.77 GB	5.2 MB	1331.7×
Ingolstadt	79.9 ms	0.5 ms	163.1×	267.3 MB	6.9 MB	38.9×
Montréal	151.3 ms	0.6 ms	245.5×	445.5 MB	5.7 MB	78.6×
NYC	1.78 s	42.9 ms	41.5×	2.50 GB	5.0 MB	510.7×
Rotterdam	67.6 ms	1.3 ms	53.5×	265.1 MB	4.4 MB	60.4×
Vienna	82.0 ms	1.9 ms	42.9×	239.8 MB	5.2 MB	46.2×
Zürich	5.80 s	151.9 ms	38.2×	6.97 GB	5.1 MB	1409.0×
PLATEAU (Building)	2.37 s	32.5 ms	72.7×	3.76 GB	64.4 MB	58.4×
PLATEAU (Bridge)	177.0 ms	0.3 ms	541.8×	500.2 MB	12.0 MB	41.9×
PLATEAU (Railway)	80.5 ms	2.0 ms	39.4×	294.0 MB	5.1 MB	58.1×
PLATEAU (Transport)	603 ms	14 ms	43.1×	1.0 GB	23.0 MB	46.0×
PLATEAU (Tunnels)	251 ms	2 ms	125.5×	618.4 MB	14.2 MB	43.5×
PLATEAU (Vegetation)	2.07 s	33 ms	65.3×	3.99 GB	70.4 MB	71.8×

^a Ratio = BSON metric / FlatCityBuf metric (higher values indicate better FlatCityBuf performance)

smaller datasets like PLATEAU bridge model reflects FlatCityBuf’s zero-copy deserialization advantage, where parsing overhead represents a larger proportion of total processing time.

Memory consumption was reduced by factors ranging from 38.9× to 1409.0×, with the largest improvements seen in datasets such as Zürich and Helsinki. This substantial memory efficiency stems from BSON’s requirement to deserialize the entire document into memory structures, while FlatCityBuf enables direct access to data without full memory allocation. The performance gains are generally more pronounced than those observed in the CBOR comparison.

5.4.6 Summary of local environment benchmark

To summarise the results of the local environment benchmark, FlatCityBuf demonstrates significant performance improvements across all tested formats and datasets, with varying degrees of enhancement depending on the comparison format and dataset characteristics.

- **Processing time:** Processing time improvements represent the primary objective of this research. Compared to CityJSONSeq, FlatCityBuf achieved speedups ranging from 8.6× (3DBAG) to 256.8× (PLATEAU bridge model). Against CBOR, improvements ranged from 11.2× (3DBAG) to 194.7× (PLATEAU bridge model). For BSON, the most dramatic gains were observed, ranging from 17.8× (3DBAG) to 541.8× (PLATEAU bridge model). Generally, smaller datasets exhibit higher performance ratios, while larger datasets show significant absolute time savings despite lower ratios.
- **Memory consumption:** Memory efficiency improvements varied significantly across formats. Compared to CityJSONSeq, FlatCityBuf achieved reductions ranging from 1.9× (Rotterdam) to 6.3× (PLATEAU bridge model). Against CBOR, memory consumption was reduced by factors of 27.3× to 1011.2×, with the largest improvements in datasets like Helsinki and Zürich. For BSON, memory reductions ranged from 38.9× to 1409.0×, with exceptional efficiency gains in large datasets. It should be noted that memory consumption comparisons with CBOR and BSON formats are not entirely equitable, as these formats require encoding and loading entire datasets into memory, while both FlatCityBuf and CityJSONSeq support streaming operations that enable incremental data processing without full memory allocation.

5.5 Benchmark over the web

To evaluate FlatCityBuf’s performance in real-world web scenarios, we compared it with the 3DBAG **api!** (?). The 3DBAG **api!** currently supports two query types: *feature ID query* for retrieving CityJSONFeature by identifier (e.g., `identificatie` attribute of 3DBAG) and *bounding box query* for spatial queries with configurable result limits via the `limit` parameter.

While network-based benchmarking provides more realistic performance insights, it introduces additional complexity due to variable network latency and server-side factors. As discussed in ??, FlatCityBuf operates without server-side processing, requiring only static file storage, contrasting with traditional application and database servers.

We acknowledge that this comparison is not entirely equitable due to fundamental architectural differences and the 3DBAG **api!** being a public service potentially handling concurrent requests. However, this comparison remains valuable as API-based access represents the current standard approach for CityJSON data consumption in web applications.

5.5.1 Benchmark environment

For the web-based benchmark, we used the 3DBAG dataset. The FlatCityBuf implementation utilised a static file encoding the entire Netherlands dataset, resulting in a 70.4 GB file containing all features, attribute indices for all attributes, and spatial indexing. Data retrieval was performed using a Rust program with **http!** range requests (browser-based testing was avoided due to disk caching effects). The 3DBAG **api!**, publicly available at <https://api.3dbag.nl/>, operates on a Flask backend with PostgreSQL and PostGIS extension for database management (?).

To account for network variability and potential outliers, we collected 100 samples for each method with 10 warmup samples.

5.5.2 Feature ID query

Both FlatCityBuf files and the 3DBAG **api!** database organise features according to technical implementation decisions (e.g., FlatCityBuf features are typically sorted by Hilbert curve). To ensure fair comparison by identifier, we selected 5 features distributed across different regions of the Netherlands, representing landmark or well-known buildings. The benchmark results represent the average performance across 100 samples for all 5 features.

?? presents the performance comparison between FlatCityBuf and the 3DBAG **api!** for feature ID queries. Overall, FlatCityBuf demonstrates approximately $2.1\times$ faster performance than the 3DBAG **api!** for identifier-based feature retrieval. Performance ranged from $2.7\times$ faster (Groningen station) to $1.5\times$ faster (Eindhoven station).

Table 5.9: Feature ID query performance comparison between FlatCityBuf and 3DBAG API

Feature ID	Location	FCB (ms)	API (ms)	Ratio
NL.IMBAG.Pand.0503100000032914	TU Delft BK building	935.8	2412.5	2.6×
NL.IMBAG.Pand.0363100012185598	Amsterdam Central Station	858.0	2106.7	2.5×
NL.IMBAG.Pand.0014100010938997	Groningen Station	821.7	2254.8	2.7×
NL.IMBAG.Pand.0772100000295227	Eindhoven Station	1378.7	2013.4	1.5×
NL.IMBAG.Pand.0153100000261851	Enschede Station	1070.4	2058.8	1.9×
Average		1012.9	2169.2	2.1×

FCB = FlatCityBuf

API = 3DBAG **api!**Ratio = 3DBAG **api!** / FlatCityBuf (higher values indicate better FlatCityBuf performance)

5.5.3 Bounding box query

For spatial query performance comparison, we selected a $2\text{km} \times 2\text{km}$ bounding box around the Delft University of Technology campus, requesting the first 10 features within the area (matching the 3DBAG **api!**'s default limit). The benchmark results represent the average of 100 samples. The bounding box coordinates were (84000.0, 444000.0, 86000.0, 446000.0) in the Amersfoort / RD New + NAP height (EPSG:7415) coordinate system.

FlatCityBuf demonstrated approximately $15.1\times$ faster performance than the 3DBAG **api!** for bounding box queries. This significant improvement over feature ID queries likely results from FlatCityBuf's Hilbert curve-based feature sorting, which enables spatially proximate features to be retrieved in batched operations.

Table 5.10: Bounding box query performance comparison between FlatCityBuf and 3DBAG API

Query Type	FlatCityBuf (ms)	3DBAG API (ms)	Ratio
Bounding box ($2\text{km} \times 2\text{km}$)	492.6	7420.3	15.1×

Ratio = 3DBAG API / FlatCityBuf (higher values indicate better FlatCityBuf performance)

Despite the architectural differences between FlatCityBuf and the 3DBAG API, FlatCityBuf demonstrated superior performance across both query patterns. Notably, FlatCityBuf does not distinguish between identifier-based and attribute-based queries, as both utilize the same underlying mechanism. Consequently, we can expect similar performance characteristics for other attribute-based queries as well. These performance results highlight FlatCityBuf's potential for web application deployment.

6 Discussion

This chapter discusses the implications of our experimental results and their broader significance for 3D city modelling applications.

6.1 Use Cases of FlatCityBuf

This section examines the most appropriate application scenarios for the FlatCityBuf format based on its demonstrated performance characteristics.

6.1.1 Flexible Data Download

Providing users with the ability to download specific data of interest represents one of the most valuable applications of 3D city models, particularly within open data initiatives. Existing services such as 3DBAG offer download functionality for CityJSON data in various formats including CityJSON, OBJ, and GeoPackage. However, these services typically constrain users to downloading predefined tiles rather than precisely the data matching their specific requirements. [This web prototype](#) demonstrates that users can download precisely the data they require. This implementation successfully showcases FlatCityBuf's capability to facilitate targeted data retrieval. Through its attribute indexing mechanism, users can download filtered datasets based on specific criteria, such as features exceeding 100 metres in height.

6.1.2 Data Processing

As demonstrated by the performance benchmarks, FlatCityBuf excels in read operations compared to alternative data formats, making it particularly suitable for analysing large-scale datasets. This performance advantage is especially valuable in data processing pipelines where **io!** operations constitute a significant bottleneck.

A compelling example is the 3DBAG generation pipeline, which involves multiple stages that require reading and writing CityJSONSeq files. In such workflows, **io!** performance directly impacts overall processing time. This efficiency becomes increasingly important when processing large urban datasets containing millions of building features.

The format also simplifies analytical workflows. Conventional approaches to large-scale data processing often require chunking data across multiple files, necessitating additional programming to manage file aggregation. In contrast, FlatCityBuf encapsulates data in a single file that can be efficiently loaded and accessed, even in web-based environments, streamlining analytical processes. This unified approach reduces the complexity of data management while maintaining high performance for both selective queries and full dataset processing.

6.2 Impact on Server Architecture

FlatCityBuf introduces significant opportunities for simplifying server architectures for 3D city model delivery.

6.2.1 Traditional Server Architecture

Conventional server architectures for 3D city models typically employ both application and database servers. For example, ? utilises PostgreSQL or Oracle as the database server with PostgREST API (?) providing data access through its toolchain. Similarly, the 3DBAG **api** uses PostgreSQL as its database server and Flask (Python web framework) as the application server.

In contrast, FlatCityBuf operates as a static file, requiring only a basic **http** server such as Nginx (?) for data distribution. This approach aligns with modern cloud service offerings, where providers like AWS S3 (?) and Google Cloud Storage (?) offer optimised solutions for serving static content.

6.2.2 Cloud Architecture Advantages

Scalability

Scalability presents a significant challenge in traditional server architectures. These systems typically employ **rdbms**! (**rdbms**!) that often encounter scaling limitations. Common mitigation strategies include sharding and replication (horizontal scaling) or resource expansion (vertical scaling), both requiring additional computational, memory, and storage resources.

FlatCityBuf circumvents these challenges by functioning as a static file that can leverage cloud providers' inherent scalability and high availability infrastructure. This characteristic offers substantial benefits for applications built on 3D city model data. Service providers can host static FlatCityBuf files on standard servers, allowing unrestricted access for various use cases without implementing the rate-limiting mechanisms often necessary with traditional server architectures.

Cost-effectiveness

FlatCityBuf contributes significantly to operational cost-effectiveness. Although precise server costs vary according to specific use cases, hosting static files through cloud service providers is generally substantially more economical than maintaining dedicated database and application servers. For example, Google Cloud's storage service, Cloud Storage, costs \$0.020 USD per GB per month in the Netherlands (europe-west4) region¹. In contrast, computing services such as Compute Engine cost \$0.25 USD per vCPU hour for on-demand instances in the same region (4 vCPU, 16 GiB memory, 375 GiB SSD)². While direct cost comparisons between storage and compute services are complex due to their different pricing models, storage services offer inherently unlimited scalability, whereas compute services require provisioning additional

¹Google Cloud Storage Pricing, <https://cloud.google.com/storage/pricing#europe>, accessed January 2025

²Google Cloud Compute Engine Pricing, <https://cloud.google.com/compute/all-pricing?hl=en>, accessed January 2025

server instances to achieve substantial scalability. This fundamental difference in architecture demonstrates that hosting static files is considerably more cost-effective than traditional server architectures requiring continuous compute resources.

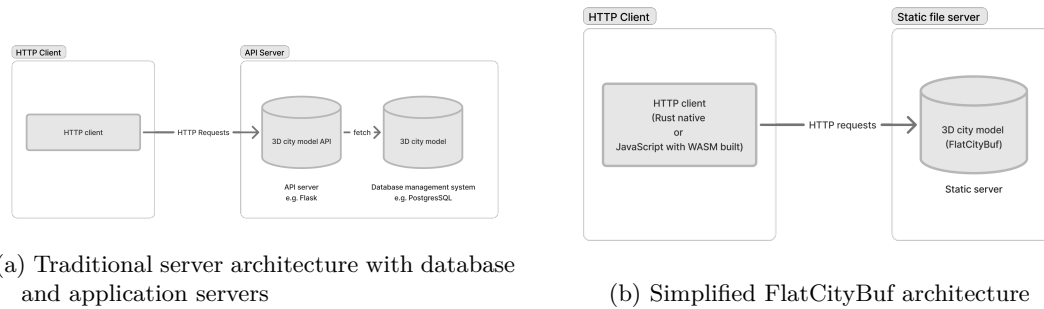


Figure 6.1: Comparison between traditional and FlatCityBuf server architectures. The proposed method eliminates the need for complex database infrastructure by leveraging static file hosting with built-in spatial and attribute indices.

6.3 Limitations

Despite its advantages in simplicity, scalability, and cost-effectiveness, FlatCityBuf does present certain limitations that warrant consideration.

6.3.1 Query Flexibility

While FlatCityBuf supports both spatial and attribute indexing, its query capabilities remain more constrained than those of specialised spatial database applications. Traditional approaches employing **rdbs** with spatial indexing provide more comprehensive query functionality. For instance, 3DCityDB enables filtering by **lod**, CityObject type (?), and various other parameters, whereas FlatCityBuf primarily supports attribute-based filtering. Similarly, regarding spatial functions, 3DCityDB can utilise the extensive spatial capabilities of PostGIS (?), while FlatCityBuf currently only implements bounding box queries, nearest neighbour queries, and point intersection queries. Consequently, FlatCityBuf is optimised for scenarios requiring relatively straightforward filtering conditions.

6.3.2 Client-side Application Complexity

Although FlatCityBuf simplifies server architecture, it introduces additional complexity in client-side applications, which must implement logic for loading and processing the format. This shift in computational responsibility follows the client-server architecture spectrum described by ?, who categorised systems ranging from "Thin Client" (where clients primarily handle display) to "Thick Client" (where clients perform most processing tasks).

Figure ?? illustrates the original model proposed by ?, while Figure ?? demonstrates where FlatCityBuf fits within this framework. As these figures show, FlatCityBuf represents an

extreme case of the "Thick Client" architecture. Since the client assumes responsibility for filtering services in addition to other processing tasks, the complexity exceeds that of traditional architectures where such operations are handled server-side.

This architectural choice has implications for interoperability. OGC API (?) and equivalent Web **api!** services adhere to standardised designs that enable universal client access—whether through command-line interfaces, web browsers, or mobile applications. While FlatCityBuf supports cross-platform deployment, it requires language-specific or platform-specific library implementations, potentially limiting its accessibility compared to standard web APIs.

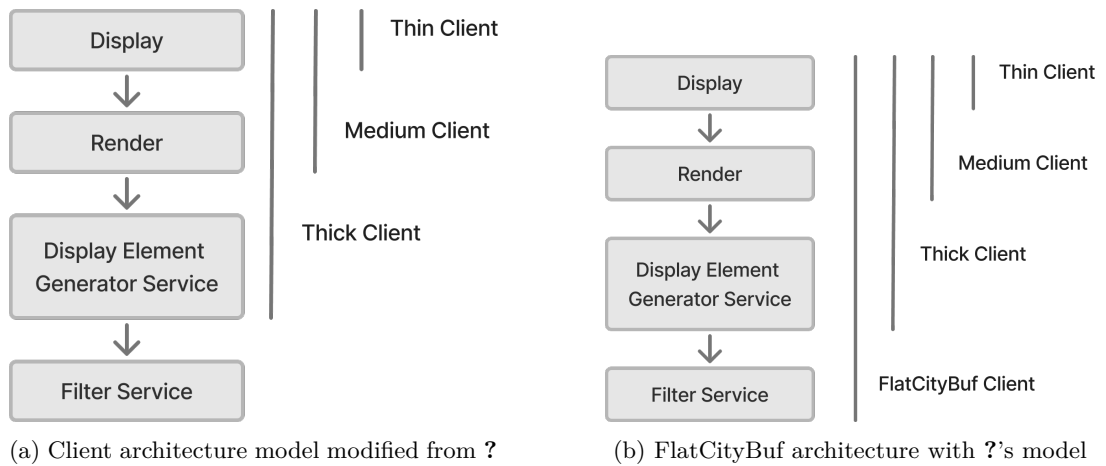


Figure 6.2: Comparison of client complexity with ?'s model and FlatCityBuf's architecture.

6.3.3 Update Complexity

Zero-copy data formats like FlatCityBuf generally present challenges for data updates due to their relatively rigid structure. Fixed-size data types such as integers or floating-point numbers cannot be dynamically converted to alternative types. Furthermore, since the format contains immutable spatial and attribute indices, updating the data necessitates rewriting the entire file. This characteristic renders FlatCityBuf less suitable for frequently updated datasets, positioning it instead as an optimal solution for data analysis and efficient download services.

7 Conclusion and Future Work

7.1 Research Summary and Limitations

This research addressed the limitations of existing 3D city model formats in cloud environments by optimising CityJSONSeq encoding through FlatCityBuf, a binary format leveraging FlatBuffers serialization with spatial and attribute indexing mechanisms.

The main contributions of this research include:

- A hierarchical FlatBuffers schema with five components (magic bytes, header section, spatial index, attribute index, and features section) enabling zero-copy access and 10-20× faster retrieval times while maintaining CityJSON compatibility
- Dual indexing mechanisms: Packed Hilbert R-tree for spatial queries with logarithmic-time retrieval, and Static B+Tree (S+Tree) for attribute-based queries supporting exact matches, ranges, and complex filtering
- **http!** Range Request optimisation through explicit file alignment boundaries, enabling efficient retrieval of specific data subsets without downloading entire datasets

Benchmarks confirmed sub-second performance even with datasets containing hundreds of thousands of features. The approach eliminates complex database infrastructure, reduces operational costs through static file hosting, and maintains fast response times across large datasets.

Despite its advantages, FlatCityBuf has several limitations. The query capabilities are more constrained than specialised spatial database applications, lacking advanced spatial operations available in systems like 3DCityDB (?) with PostGIS (?). The format introduces complexity in client-side applications requiring custom loading logic, presenting potential adoption barriers. Additionally, FlatCityBuf's rigid structure presents challenges for data updates, requiring rewriting entire files when modifying data, making it more suitable for read-intensive applications than dynamic content management systems.

7.2 Future Work

Based on the research findings and identified limitations, several promising directions for future work emerge.

Expanding language support beyond Rust would enhance the format's accessibility and ecosystem integration. For some languages, implementing binding libraries to the existing Rust implementation could provide an efficient path to broader adoption. Languages with garbage collection mechanisms—such as Python, JavaScript, and Java—present particularly interesting implementation targets. These languages manage memory differently than Rust, which could

7 Conclusion and Future Work

impact performance characteristics of zero-copy operations. Implementation in Python would enable seamless integration with geospatial analysis workflows, while JavaScript support would facilitate web-based visualisation without WebAssembly. Testing performance across these languages would provide valuable insights into optimisation strategies for different memory management approaches.

Investigating alternative serialization frameworks could reveal different efficiency patterns. Column-oriented formats like Apache Parquet warrant exploration, particularly for analytical workloads involving selective attribute access. Such formats excel at accessing specific fields across many records, potentially offering significant advantages for city-scale analytics where only certain properties (like building heights or energy consumption) are needed. Future research should quantify these trade-offs through comparative benchmarks across various query patterns and datasets sizes.

While a web prototype for FlatCityBuf exists, it currently only displays data as JSON without geometric visualisation. Developing specialised web viewers would demonstrate the format's practical benefits in interactive contexts. Progressive loading strategies could enable smooth navigation of massive datasets on bandwidth-constrained devices by initially loading lower-detail geometries and enhancing detail as users zoom, significantly improving user experience while leveraging the format's efficient partial data retrieval mechanisms.

A FlatCityBuf Schema

A.1 Header

```
1  include "geometry.fbs";
2  include "extension.fbs";
3
4
5  enum ColumnType: ubyte {
6      Byte,                // Signed 8-bit integer
7      UByte,               // Unsigned 8-bit integer
8      Bool,                // Boolean
9      Short,               // Signed 16-bit integer
10     UShort,              // Unsigned 16-bit integer
11     Int,                 // Signed 32-bit integer
12     UInt,                // Unsigned 32-bit integer
13     Long,                // Signed 64-bit integer
14     ULong,               // Unsigned 64-bit integer
15     Float,               // Single precision floating point
16     Double,              // Double precision floating point
17     String,              // UTF8 string
18     Json,                // General JSON type intended to be
19     application specific
20     DateTime,             // ISO 8601 date time
21     Binary,               // General binary type intended to be
22     application specific  // Array of values
23 }
24
25 table Column {
26     index: ushort;        // Column index (0 = first column) This
27     // index is used to identify the column in the FlatBuffer. The reason
28     // why index is used instead of column name is to save more space on
29     // attribute field.
30     name: string (required); // Column name
31     type: ColumnType;        // Column type
32     title: string;           // Column title
33     description: string;     // Column description (intended for
34     // free form long text)
35     precision: int = -1;     // Column values expected precision (-1 =
36     // unknown) as defined by SQL
37     scale: int = -1;         // Column values expected scale (-1 =
38     // unknown) as defined by SQL
39     nullable: bool = true;   // Column values expected nullability
```

A FlatCityBuf Schema

```
33     unique: bool = false;           // Column values expected uniqueness
34     primary_key: bool = false;      // Indicates this column has been (part
35     metadata: string;               // Column metadata (intended to be
    application specific and suggested to be structured fx. JSON)
36 }
37
38 table ReferenceSystem {
39     authority: string;               // Case-insensitive name of the
    defining organization e.g. EPSG or epsg (NULL = EPSG)
40     version: int;                   // Version of the Spatial Reference
    System assigned by the organization (0 = not defined)
41     code: int;                      // Numeric ID of the Spatial Reference
    System assigned by the organization (0 = unknown)
42     code_string: string;            // Text ID of the Spatial Reference
    System assigned by the organization in the (rare) case when it is not
    an integer and thus cannot be set into code
43 }
44
45 struct Vector {
46     x: double;
47     y: double;
48     z: double;
49 }
50
51 struct Transform {
52     scale: Vector;
53     translate: Vector;
54 }
55
56 struct GeographicalExtent {
57     min: Vector;
58     max: Vector;
59 }
60
61 struct AttributeIndex {
62     index: ushort;
63     length: uint;
64     branching_factor: ushort;
65     num_unique_items: uint;
66 }
67
68 struct Vec2 {
69     u: double;
70     v: double;
71 }
72
73 table Material {
74     name: string (required);
75     ambient_intensity: double = null; // from 0.0 to 1.0
    // Expected to contain 3 items from 0.0 to 1.0 for (RGB);
    implementations MUST check length.
76     diffuse_color: [double];
77     emissive_color: [double];
78 }
```

```

79     specular_color: [double];
80     shininess: double = null; // from 0.0 to 1.0
81     transparency: double = null; // from 0.0 to 1.0
82     is_smooth: bool = null;
83 }
84
85 enum TextureFormat:ubyte {
86     PNG,
87     JPG
88 }
89
90 enum WrapMode:ubyte {
91     None,
92     Wrap,
93     Mirror,
94     Clamp,
95     Border
96 }
97
98 enum TextureType:ubyte {
99     Unknown,
100     Specific,
101     Typical
102 }
103
104 table Texture {
105     type: TextureFormat; // NOTICE: "type" fields refers to TextureFormat
106     while "textureType" refers to TextureType.
107     image: string (required); // Image file name / URL
108     wrap_mode: WrapMode = null;
109     texture_type: TextureType = null; // e.g., "unknown", "specific", or
110     "typical"
111     // Expected to contain 4 items (RGBA)
112     border_color: [double]; // from 0.0 to 1.0 for (RGBA)
113 }
114
115 table Appearance {
116     materials: [Material];
117     textures: [Texture];
118     vertices_texture: [Vec2]; // List of UV coordinates, each coordinate
119     must be between 0.0 and 1.0
120     default_theme_texture: string; // Default theme name for textures
121     when multiple themes exist
122     default_theme_material: string; // Default theme name for materials
123     when multiple themes exist
124 }
125
126 struct DoubleVertex {
127     x: double;
128     y: double;
129     z: double;
130 }
131
132 table Header {

```

A FlatCityBuf Schema

```
128     transform: Transform;           // Transformation vectors
129     appearance: Appearance;         // Appearance object for
materials and textures
130     columns: [Column];              // Attribute columns schema
(can be omitted if per feature schema)
131     semantic_columns: [Column];     // Semantic columns schema (
can be omitted if per feature schema)
132     features_count: ulong;          // Number of features in
the dataset (0 = unknown)
133     index_node_size: ushort = 16;   // Index node size (0 = no
index)
134     attribute_index: [AttributeIndex];
135     // metadata
136     geographical_extent: GeographicalExtent; // Bounds
137     reference_system: ReferenceSystem;      // Spatial Reference System
138     identifier: string;                     // Dataset identifier
139     reference_date: string;                 // Reference date
140     title: string;                          // Dataset title
141     // geometry templates
142     templates: [Geometry];
143     templates_vertices: [DoubleVertex];
144     // extensions
145     extensions: [Extension];
146     // Point of contact
147     poc_contact_name: string;               // Point of contact name
148     poc_contact_type: string;               // Point of contact type
149     poc_role: string;                       // Point of contact role
150     poc_phone: string;                      // Point of contact phone
151     poc_email: string;                      // Point of contact email
152     poc_website: string;                    // Point of contact website
153     poc_address_thoroughfare_number: string; // Point of contact address
thoroughfare number
154     poc_address_thoroughfare_name: string;  // Point of contact address
thoroughfare name
155     poc_address_locality: string;            // Point of contact address
locality
156     poc_address_postcode: string;            // Point of contact address
postcode
157     poc_address_country: string;             // Point of contact address
country
158     attributes: [ubyte];                    // Other attributes that
are stored in root CityJSON object
159     version: string (required);              // CityJSON version
160 }
161
162 root_type Header;
```

Listing A.1: Header schema of FlatCityBuf

A.2 Geometry

1

```

2 enum SemanticSurfaceType:ubyte {
3     // Building
4     RoofSurface ,
5     GroundSurface ,
6     WallSurface ,
7     ClosureSurface ,
8     OuterCeilingSurface ,
9     OuterFloorSurface ,
10    Window ,
11    Door ,
12    InteriorWallSurface ,
13    CeilingSurface ,
14    FloorSurface ,
15    // WaterBody
16    WaterSurface ,
17    WaterGroundSurface ,
18    WaterClosureSurface ,
19    // Transportation ("Road", "Railway", "TransportSquare")
20    TrafficArea ,
21    AuxiliaryTrafficArea ,
22    TransportationMarking ,
23    TransportationHole ,
24
25    // Extension objects. In the JSON data, it's written like "+
    ThermalSurface". However as we can't expect the extended semantic
    surface type, just mark it as "ExtraSemanticSurface".
26    ExtraSemanticSurface
27 }
28
29 enum GeometryType:ubyte {
30     MultiPoint ,
31     MultiLineString ,
32     MultiSurface ,
33     CompositeSurface ,
34     Solid ,
35     MultiSolid ,
36     CompositeSolid ,
37     GeometryInstance
38 }
39
40 table MaterialMapping {
41     theme: string;
42     solids: [uint];
43     shells: [uint];
44     vertices: [uint]; // flat list of vertex indices.
45     // The depth of material indices will be boundaries depth minus 2:
46     // - For MultiSurface/CompositeSurface: one material index per surface
47     // - For Solid: one material index per surface in each shell
48     // - For MultiSolid/CompositeSolid: one material index per surface in
    each shell of each solid
49     value: uint = null; // used only when it uses the shared material of
    CityJSON
50 }
51

```

```

52 table TextureMapping {
53     theme: string;
54     solids: [uint];
55     shells: [uint];
56     surfaces: [uint];
57     strings: [uint];
58     vertices: [uint]; // flat list of vertex indices.
59     // The depth of texture indices matches the boundaries array:
60     // - For each ring: first vertex is texture index, remaining vertices
        are UV coordinate indices
61     // - UV coordinates must be between 0.0 and 1.0 and reference
        vertices_texture array
62 }
63
64 table Geometry {
65     type: GeometryType;
66     lod: string;
67
68     // these are lengths in the
69     // depending on the geometry_type, different fields are used
70     solids: [uint];
71     shells: [uint];
72     surfaces: [uint];
73     strings: [uint]; // Rings or LineStrings
74
75     boundaries: [uint]; // flat list of vertex indices
76
77     semantics: [uint]; // flat list of semantic object indices
78     semantics_objects: [SemanticObject];
79
80     material: [MaterialMapping]; // Maps each surface/shells to an index in
        appearance.materials.
81     texture: [TextureMapping]; // Maps each primitives to an index in
        appearance.textures.
82 }
83
84 table SemanticObject {
85     type: SemanticSurfaceType;
86     attributes: [ubyte];
87     children: [uint];
88     parent: uint = null; // default is null, important to be able to
        check if this field is set
89     extension_type: string; // extension type of the semantic object. e.g.
        "+ThermalSurface"
90 }
91
92 struct TransformationMatrix {
93     m00: double;
94     m01: double;
95     m02: double;
96     m03: double;
97     m10: double;
98     m11: double;
99     m12: double;

```

```

100 m13:double;
101 m20:double;
102 m21:double;
103 m22:double;
104 m23:double;
105 m30:double;
106 m31:double;
107 m32:double;
108 m33:double;
109 }
110
111 table GeometryInstance {
112     // "type": "GeometryInstance" isn't written in the file as it's obvious
113     transformation: TransformationMatrix;
114     template: uint;
115     boundaries: [uint]; //contains only one vertex index of vertices. "
116     referencePoint" of CityGML

```

Listing A.2: Geometry schema of FlatCityBuf

A.3 Extension

```

1 // Extension is a struct that contains schema of the extension.
2 // To simplify FlatBuffers schema, we just store stringified JSON schema.
3 // This Extension can be derived from ExtensionMeta's url.
4 table Extension {
5     // "type": "CityJSONExtension" isn't written in the file as it's
6     // obvious
7     name: string; // name of the extension. It's the same as ExtensionMeta's
8     // name.
9     description: string; // description of the extension. It's the same as
10    // ExtensionMeta's description.
11    url: string; // url of the extension. It's the same as ExtensionMeta's
12    // url.
13    version: string; // version of the extension. It's the same as
14    // ExtensionMeta's version.
15    version_cityjson: string; // version of the extension in CityJSON
16    // format.
17    extra_attributes: string; // extra attributes of the extension.
18    // stringified JSON object.
19    extra_city_objects: string; // extra city objects of the extension.
20    // stringified JSON object.
21    extra_root_properties: string; // extra root properties of the
22    // extension. stringified JSON object.
23    extra_semantic_surfaces: string; // extra semantic surfaces of the
24    // extension. stringified JSON object.
25 }

```

Listing A.3: Extension schema of FlatCityBuf

A.4 Feature

```
1
2  include "header.fbs";
3  include "geometry.fbs";
4
5  // namespace FlatCityBuf;
6
7  enum CityObjectType:ubyte {
8      Bridge,
9      BridgePart,
10     BridgeInstallation,
11     BridgeConstructiveElement,
12     BridgeRoom,
13     BridgeFurniture,
14
15     Building,
16     BuildingPart,
17     BuildingInstallation,
18     BuildingConstructiveElement,
19     BuildingFurniture,
20     BuildingStorey,
21     BuildingRoom,
22     BuildingUnit,
23
24     CityFurniture,
25     CityObjectGroup,
26     GenericCityObject,
27     LandUse,
28     OtherConstruction,
29     PlantCover,
30     SolitaryVegetationObject,
31     TINRelief,
32
33     // Transportation objects
34     Road,
35     Railway,
36     Waterway,
37     TransportSquare,
38
39     Tunnel,
40     TunnelPart,
41     TunnelInstallation,
42     TunnelConstructiveElement,
43     TunnelHollowSpace,
44     TunnelFurniture,
45
46     WaterBody,
47
48     // Extension objects. Since we can't expect the extended city object
49     // type, just mark it as "ExtensionObject".
50     ExtensionObject
51 }
```

```

52 struct Vertex {
53     x:int;
54     y:int;
55     z:int;
56 }
57
58 table CityFeature {
59     id:string (key, required);
60     objects:[CityObject];
61     vertices:[Vertex];
62     appearance: Appearance;
63 }
64
65 table CityObject {
66     type:CityObjectType;
67     extension_type: string; // extension type of the city object. e.g. +
68                             NoiseCityFurnitureSegment
69     id:string (key, required);
70     geographical_extent:GeographicalExtent;
71     geometry:[Geometry];
72     geometry_instances:[GeometryInstance];
73     attributes:[ubyte];
74     columns:[Column];          // Attribute columns schema (optional, if
75     set it should be used instead of the header columns)
76     children:[string];
77     children_roles:[string];   // for CityObjectGroup only
78     parents:[string];
79 }
80
81 root_type CityFeature;

```

Listing A.4: Feature schema of FlatCityBuf

Colophon

This document was typeset using L^AT_EX, using the KOMA-Script class `scrbook`. The main font is Palatino.

